

**CAREER GUIDANCE - AN INTELLIGENT ANDROID
APPLICATION FOR PERSONALIZED CAREER
RECOMMENDATIONS**

Dipendra Dahal
Roshan Kandel
Shankar Jaiswal
Bachelor's Thesis
Term year Spring 2026
Degree Programme in Information Technology
Oulu University of Applied Sciences

Abstract

Oulu University of Applied Sciences
Degree Programme in Information Technology (IT)
Option of Software Development

Author(s):
Dipendra Dahal
Roshan Kandel
Shankar Jaiswal

Title of thesis: Career Guidance – An Intelligent Android Application for Personalized Career Recommendations

Term and year of completion: Spring 2026
Pages: 88

The thesis implemented an Android mobile application project, Career Guidance App, that combined role based workflows, cloud-backed profile, CV handling and AI-driven matching recommendations. The system applied an explainable CV-to-skill extraction and role-skill matching pipeline to generate readiness scores and skill-gap recommendations, and it ensured transparent, auditable decision support within a secure MVVM based architecture.

Role based features; applicant and company dashboards, controlled job posting and application workflows, and secure CV upload handling; were put into place to assist structured usage without making decisions less reliable while AI components generated explainable matching results, readiness scores and skill-gap recommendations as strictly optional decision support outputs. The Android application was built using Kotlin with Jetpack Compose and MVVM with Firebase cloud backend services to assure scalable server side logic, valid authentication and secure data handling. Supabase was used to store CV.

Informal user testing by developers stated that the role based guidance flow and explainable AI matching recommendation improved user engagement and increased job market inquiries. The findings indicated that simple and understandable decision support mechanisms could efficiently aid career planning and could be incorporated into development of Android and cloud-based job market environments.

Contents

ABSTRACT	2
CONTENTS.....	3
GLOSSARY.....	6
1 INTRODUCTION	11
1.1 Background	11
1.2 Problem Statements	14
1.3 Objectives of Research	15
1.4 Research Questions	16
1.5 Study Scope	18
2 METHODOLOGY	19
2.1 Research Approach and Design.....	19
2.2 System Architecture and Technology	21
2.3 Role Based Career Guidance Approach	24
2.4 Authentication and Access Control Strategy	25
2.5 Cloud Based Backend Design.....	27
2.6 Data Collection	29
2.7 Experimental Design and Process	31
2.8 Data Analysis	34
2.9 Trustworthiness and Validity Factors.....	36
2.10 Data Protection and Ethical Factors	37
2.11 Limitations of Methodology	39
3 SYSTEM DESIGN AND IMPLEMENTATION.....	41
3.1 System Design and Development Structure.....	41
3.2 Complete System Architecture	43
3.2.1 Design Logic.....	44
3.2.2 High Level Architecture	45
3.2.3 Workflow for Request Response	47
3.3 Design of Data Model and Schema	50
3.3.1 User Profile and Identification.....	51

3.3.2	Job Records and Application	52
3.3.3	CV Storage References.....	53
3.3.4	AI Decision Support State	53
3.4	Core Role Based Workflow Logic.....	54
3.4.1	Role Resolution and Home Dashboard	57
3.4.2	Job Listing Rules	57
3.4.3	Application Rules and Validation	57
3.5	Job Posting and Applicant Review Implementation	58
3.5.1	Job Posting Flow	58
3.5.2	Application Submission	59
3.5.3	Applicant Review for Companies.....	59
3.6	CV Upload and Document Handling Implementation	59
3.6.1	CV Upload Procedure.....	59
3.6.2	Using CV Reference in Applications.....	60
3.7	Access Control and Data Security.....	61
3.7.1	Summary of Security Architecture	61
3.7.2	CV Storage Sensitivity.....	62
3.8	Implementation of Frontend and User Interface Logic.....	62
3.8.1	Routing and Navigation	62
3.8.2	UI Design Approach	63
3.8.3	Profile Editing Strategy	63
3.9	Backend Integration and Cloud Service Logic.....	64
3.9.1	Firebase Authentication and Firestore Operations	64
3.9.2	Supabase Storage Operations	65
3.10	AI Decision Support Implementation	65
3.10.1	AI pipeline and Exported Artefacts	65
3.10.2	Role Catalogue, Matching and Readiness Scoring	66
3.10.3	CV-to-Skill Extraction and Normalization.....	66
3.10.4	User Facing AI Outputs and Persistence in Firestore.....	66
4	RESULTS	68
4.1	Working Functions.....	68
4.2	AI Decision Support Behaviour and Action Plan Engine	70
4.3	Role Selection	71
4.4	Real World Observation from Alpha Testing	73

4.5	User Experience and Outcomes.....	75
5	DISCUSSION	76
5.1	Discussion in Relation to Research Questions.....	76
5.2	Critical Evaluation and Limitations.....	79
5.3	Ethical and Data Protection Considerations	80
5.4	Practical Implications.....	81
5.5	Conclusion	82
5.6	Future Work.....	82
	REFERENCES.....	84

Glossary

Concept	Definition
ADDIE	Analysis, Design, Development, Implementation, Evaluation. It is used as an “ADDIE-inspired” lifecycle to structure how the Android system and the AI pipeline were planned, built and tested.
AI	Artificial Intelligence. In the thesis, AI is used to analyse CV information and produce decision support outputs such as extracted skills, matching results, readiness scoring and recommendations.
Android	A mobile operating system and development platform used to build mobile applications. It is used as the main platform for implementing the career guidance application
Android Studio	The Integrated Development Environment commonly used for Android development. It is used to develop, run and debug the Android application.
API	Application Programming Interface. It is used to enable communication between software components or services when data or functionality must be accessed programmatically.
APK	Android Package Kit. It is used as the installable file format for distributing and testing an Android application on devices.
Artefacts	Outputs produced during development that are reused in the system. It is used to describe the AI-related exported resources included in the project to support matching, scoring, and recommendations.

Authentication	The process of verifying a user's identity (for example, signing in) to ensure that users with valid credentials can access the application and that actions are linked to the correct account.
Authorization	The process of controlling what an authenticated user is allowed to do. It is used to enforce different permissions for applicant and company users and to protect sensitive data.
Cloud Firestore	A cloud-hosted NoSQL document database that stores structured data like user profiles, roles, job postings, and applications.
Cloud Service	Managed online services that provide backend functionality such as databases, authentication, and storage. It is used to support scalability, reliability and centralized access control.
Compose client layer	The UI-facing layer built with Jetpack Compose. It is used to handle interface rendering and user interaction while keeping backend operations in other layers.
Compose Navigation	A navigation framework used with Jetpack Compose to move between screens. It is used to implement screen flow and role based routing inside the app.
CV	Curriculum Vitae. It is used as the main input document describing an applicant's skills and experience, and it is processed for text extraction and skill extraction.
Decision support	Support that helps users make better decisions by providing structured insights and recommendations. It is used to frame the system as guidance-oriented rather than fully automated decision-making.

Explainable AI	Artificial Intelligence that provides understandable reasons behind its outputs. It is used to present results like matched skills, missing skills, and readiness scoring in a transparent way.
Firebase	A cloud platform that provides managed backend services for apps. It is used to support authentication and database functions through SDK integration.
Firestore	A cloud NoSQL document database service (often referred to as Cloud Firestore). It is used to store application data using collections and documents.
HTTP	Hypertext Transfer Protocol. It is used for network communication, including operations such as uploading files and interacting with online services
Jetpack Compose	A modern Android UI toolkit for building screens that uses a declarative approach. It is used to implement the application interface and user workflows.
Kotlin	A programming language commonly used for Android development. It is used to implement the Android application codebase.
Logcat	Android Studio's logging tool that shows runtime messages, errors, and crashes. It is used to observe app behaviour during development and testing.
MVVM	Model-View-ViewModel. It is used as the application architecture to separate UI rendering from state/logic and data handling, improving maintainability.
NoSQL	A database model that stores data in flexible document-like structures rather than relational tables

OKHttp	An HTTP client library for Android. It is used to perform network operations such as file uploads.
PDF	Portable Document Format. It is used as the CV file format because it is widely supported and suitable for storage and text extraction.
Readiness Score	A score that summarises how well an applicant aligns with a target role. It is used to present matching results in a simple, interpretable form.
Role Based Access	A system design where access to screens and actions depends on the user's role. It is used to separate applicant workflows from company workflows.
Role-skill Matching	A matching method that compares role requirements with an applicant's extracted skills. It is used to generate explainable matching outputs and readiness scoring.
Scenario-Based	An evaluation approach based on realistic user scenarios and end-to-end workflows. It is used to test the system under practical interaction conditions.
SDK	Software Development Kit. It is used to refer to libraries and tools that enable development and service integration.
Skill Extraction	Extracting skill information from CV text and converting it into structured signals. It is used to support matching, scoring, and recommendations
Skill Gap	A missing or weak skill compared with what a target role requires. It is used to explain why a match is lower and what should be improved.

Supabase	A cloud platform that provides backend services such as storage. It is used as part of the implementation approach for handling uploaded CV files.
Text Extraction	Converting document content (such as a PDF) into readable text. It is used as a step before skill extraction and matching.
UI	User Interface. UI is used to describe the app screens and interactive elements used by applicants and companies.
URL	Uniform Resource Locator. It is used to represent a link to an online resource such as an uploaded CV document.

1 INTRODUCTION

1.1 Background

Career guidance and recruitment have become progressively complex in recent years due to rapid technological development [1], globalization and broader labour-market transformation changes in workforce expectations [2]. Applicants are no longer evaluated only based on educational background or years of experience instead employers often consider a wide range of factors like skills [2][3], adaptability, digital competence [4] and professional presentation [5]. Applicants must manage their professional information carefully to stay competitive in the job market [4].

From applicant's perspective, career development involves more than simply submitting resumes. Applicants must decide how to present themselves, which skills to highlight and how to maintain updated profiles across multiple platforms [6][7]. This process can be confusing and time consuming, especially for early career individuals or those unfamiliar with digital recruitment systems [8]. Without proper guidance or structured platforms, applicants may struggle to understand how their profiles are perceived by employers [7].

Companies face equally significant challenges. Recruitment processes often involve reviewing large numbers of applications, many of which contain incomplete or inconsistent information. Manual screening is time consuming and prone to human error [9]. On top of that companies must ensure that recruitment processes are fair, transparent and compliant with data protection regulations [10]. These challenges increase the need for structured digital systems that support efficient candidate evaluation.

The widespread adoption of smartphones has transformed how individuals interact with digital services. Mobile applications have become the primary medium for communication, information management and service access. In the context of career guidance and recruitment, mobile platforms provide continuous availability and allow users to manage professional information anytime and

anywhere [11]. Android is one of the most broadly used mobile operating systems worldwide which offers a practical platform for developing applications.

Alongside mobile development, cloud-based backend services have changed how applications are built and maintained. Traditional server-based systems require extensive configuration, maintenance and scaling effort. Cloud platforms such as Firebase provide ready-made services for authentication, database management and access control, reducing development complexity while maintaining security and scalability [12]. These technologies make it feasible for small development teams and student projects to build reliable systems with real-world relevance.

AI-driven decision support [13] emerged as a promising approach for improving decision making in complex, information rich situations, where people must weigh many factors under time and attention limits. Research on decision support systems shows that well-designed support can improve decision strategies and performance by summarizing relevant information and reducing unnecessary mental effort during the decision process. AI-driven decision support therefore requires alignment with the user's real goal which supports the decision but not replace it. Clear and understandable explanations that help users judge when and why a recommendation applies and meaningful human oversight [14] so users can question, override and correct outputs when needed.

Integrating Artificial Intelligence (AI) into career guidance and decision-support systems offers a significant opportunity to improve both personalization and clarity in career planning [15]. AI-driven decision support can help users identify suitable career goals by analysing their profile signals such as skills, education, and experience, and then matching them to role requirements to suggest the most aligned pathways. This enables automated skill gap detection, goal specific recommendations, and ranked learning priorities that guide users toward realistic next steps [16]. By combining role skill matching scores with transparent explanations and actionable learning plans, AI-driven decision support can turn career exploration into a structured, personalized process where users receive goal recommendations that fit user's current background and show exactly what to improve to reach the target role.

With recent advances in AI-driven decision support [13], particularly explainable person-job recommendation methods [17], skill-driven job transition modelling [16], and interpretable “what to learn next” skill recommendation techniques [18], building career guidance platforms that tailor career goal suggestions to a user’s current skills and experience has gotten increasingly feasible and scalable. The increasing accessibility and availability of deployable AI models and APIs makes it viable to deliver complete, near real time explanations and suggestions, helping users understand why a role is recommended, what skill gaps matter most, and what practical next steps can improve alignment with a target career goal [13][18].

The thesis progresses on research opportunity by building and implementing a mobile-based, AI-driven career guidance application with role based access for applicants and companies. The source code, system design, and data models convey a research informed implementation that executes AI-driven decision support through skill extraction, role-skill matching, and transparent recommendations. The application introduced a comprehensive system comprising:

- Role based access with separate applicant and company workflows
- A resume (CV) intake pipeline with PDF upload, text extraction, and profile structuring
- A career-goal recommendation and role–skill matching engine
- Automated skill-gap detection with ranked learning priorities
- A readiness scoring mechanism that summarizes alignment to a target role
- An actionable learning plan with progress tracking and completion updates
- A job posting and application module linking applicants, jobs, and application history

By integrating these features, the system helps users convert scattered career information into a structured decision-making process, where recommendations are explained and tied to practical next steps. The objective was not to replace career decisions with automated outcomes, but to support informed goal selection, reduce cognitive overload, strengthen self-awareness of skill gaps, and enable sustained, measurable progress toward a chosen career pathway.

1.2 Problem Statements

In spite of the availability of numerous online and computer based recruitment systems, several limitations persist like many systems focus heavily on web-based interfaces, offering limited functionality or poor usability on mobile devices. This creates accessibility issues for users who rely primarily on smartphones for digital interactions.

Applicants often lack insight into how their profiles are interpreted by employers. Information may be scattered across multiple platforms, leading to inconsistencies and outdated data. Applicants may not clearly understand which profile elements are most relevant to companies, resulting in ineffective self-presentation. Current platforms rarely provide AI-driven, explainable decision support that can translate a CV into structured skills, compare those skills against job or role requirements, and clearly explain why a role is a good match or not [17]. Even when AI is used, recommendations are often opaque or generic, without transparent readiness scoring, prioritized skill-gap identification, and actionable learning steps. This lack of interpretable role skill matching and guided improvement pathways limits both applicants who need clear, personalized next steps and companies who need consistent, scalable screening support with understandable reasoning.

From the company perspective, recruitment systems often present large volumes of unstructured applicant data. Employers must manually filter profiles, review documents and assess suitability without sufficient system support. This increases workload and can reduce the effectiveness of the recruitment process.

Developing a system that supports multiple user roles introduces challenges related to authentication, authorization and data security. Without clear role based access control, systems risk exposing sensitive data or allowing unauthorized actions. Applications that lack a structured architectural approach can become difficult to maintain, extend or debug. Adding AI-based functionality introduces further complexity because the system must securely process

sensitive CV content, extract structured profile data like skills and experience, and generate role–skill matching results without leaking personal information or producing inconsistent outputs. This requires controlled access to AI-generated insights so that applicants and companies only see data and recommendations appropriate to their role, along with mechanisms to keep model outputs traceable and explainable. It also increases the need for a well-structured architecture such as MVVM with clear separation of UI, domain, and data layers so the AI pipeline, cloud services, and mobile app logic remain maintainable, testable and scalable [19].

This thesis answers these challenges by designing and implementing a mobile-based career guidance android app that supports structured role based access, secure authentication, reliable data storage and a maintainable software architecture, Integration of an explainable AI pipeline for CV-to-skill extraction, role skill matching with readiness scoring, and transparent skill-gap recommendations with actionable learning priorities.

1.3 Objectives of Research

The major objective of this thesis is to design and implement a practical career guidance mobile application using modern Android development tools and cloud backend services. This objective emphasizes hands on implementation rather than theoretical modelling. The thesis aims to implement an AI-driven decision-support pipeline that processes uploaded CV documents, extracts structured profile information such as skills and experience, and performs explainable role–skill matching to produce readiness scores, prioritized skill-gap detection, and actionable learning recommendations that guide applicants toward realistic career goals while supporting consistent, scalable evaluation for companies [17].

One objective of the thesis is to design a role based system that distinguishes clearly between applicants and companies. This objective ensures that each user group interacts with the system in an appropriate and secure manner. By implementing role based access, the system prevents unauthorized actions and improves usability by presenting only relevant features to each user.

Another objective is to implement secure user authentication and access control using cloud-based services. Authentication is a fundamental requirement for systems that handle personal data. This objective focuses on protecting user identity, securing access to stored data and ensuring compliance with basic data protection principles.

A further objective is to develop profile management functionality using a scalable cloud database. This includes storing user information, supporting updates and ensuring data consistency across sessions. Cloud databases enable real-time synchronization, which improves user experience and system responsiveness.

The thesis aims to apply the Model-View-ViewModel (MVVM) architectural pattern to structure the Android app. This objective focuses on improving code organization, deducting coupling between components to enable easier testing and future development. A maintainable architecture is essential for systems expected to evolve over time.

Another objective of the thesis is to implement an explainable AI-driven decision-support pipeline that transforms uploaded CV documents into structured profile data and uses this information to support career guidance and recruitment workflows. This objective focuses on extracting relevant skills and experience signals, matching applicant profiles against job or role requirements, and generating interpretable outputs such as readiness scores, prioritized skill-gap identification, and actionable learning recommendations. By integrating transparent AI reasoning into the mobile application, the system improves clarity for applicants on how their profiles align with target roles and supports companies with more consistent, scalable evaluation of candidate suitability.

1.4 Research Questions

This thesis is guided by three research questions that examine both the design and effectiveness of the proposed system. The research questions ensure that the thesis evaluates not only whether the system works, but also whether the chosen design decisions are appropriate.

Research Question 1 (RQ1)

How can a role based career guidance system be effectively implemented as an Android mobile application using cloud backend services and an integrated AI decision-support pipeline?

This question explores the feasibility of implementing a multi-role system (applicant and company) within a mobile application while also supporting CV intake, profile structuring, and AI-driven guidance. It focuses on how Android development and cloud services can support secure authentication, role separation, PDF-based CV upload and processing, and structured data handling. Addressing this question helps evaluate whether the chosen technologies provide sufficient support for building a practical career guidance system that delivers explainable role–skill matching and recommendations within a mobile workflow [20][21].

Research Question 2 (RQ2)

How does cloud-based authentication and database management contribute to security, scalability, usability and controlled access to AI-generated insights in a career guidance application?

This question examines the role of cloud backend services in managing sensitive user data and AI-related outputs. It evaluates how authentication mechanisms protect user accounts, how role based rules enforce access control, and how cloud infrastructure supports scalability for profile data, job posting data, and application history. The question also considers usability aspects, such as login experience, real-time data updates, and ensuring that AI-generated readiness scores, skill-gap results, and recommendations are visible only to the appropriate user role [20][21][22].

Research Question 3 (RQ3)

How does the MVVM architectural pattern support maintainability, clarity and reliable integration of AI-driven features in a role based Android application?

This question investigates the impact of architectural design on software quality when the application includes both standard app functionality and AI-supported workflows. It examines how separating user interface logic, business logic, data management, and AI interaction improves code readability, reduces complexity, and supports future enhancements. This question is important for evaluating the long-term sustainability of the application, including maintainable integration of CV processing, role–skill matching results, explainable outputs, and progress tracking features in a structured Android codebase [23] [17].

1.5 Study Scope

The scope of this thesis is limited to the design and implementation of a mobile application developed for the Android platform. The system supports two user roles: applicant and company. Backend functionality includes authentication, cloud database management and file storage. Implementing an AI-driven decision-support pipeline that processes uploaded CV documents, performs CV-to-skill extraction and profile structuring, and generates explainable role-skill matching outputs such as readiness scoring, prioritized skill-gap identification and actionable learning recommendations.

Features such as real-time messaging, interview scheduling, automated candidate ranking or cross-platform deployment are outside the scope of this study. These features are acknowledged as potential future extensions.

2 METHODOLOGY

This chapter explains the methodological approach used in this thesis. It describes how the Career Guidance application was designed, developed and evaluated as a practical software project including the AI-driven decision-support pipeline used for CV-to-skill extraction, role-skill matching, readiness scoring and skill-gap based learning recommendations. The chapter focuses on the research approach, system design methodology and the reasoning behind key technical and architectural decisions, including how the AI component was implemented using the project AI pipeline artifact and integrated into the Android application to produce transparent, explainable outputs for applicants and companies. The objective of this chapter is to clarify how the work was carried out and to demonstrate that the chosen methods are appropriate for achieving the research objectives defined in Chapter 1.

2.1 Research Approach and Design

The thesis is oriented by design and implementation led by research approach, like methodologies commonly used in applied educational and system development studies. The research is grounded in the practical design and development of a mobile-based career guidance application rather than in theoretical model formulation. The study follows an applied research paradigm, aiming to address real-world challenges faced by job applicants and companies in modern recruitment environments using Android mobile technology and cloud backend services.

The research process is structured using a development lifecycle approach inspired by the ADDIE model [24], which is widely applied in the design of digital systems and learning tools. The stages include analysis, design, development, implementation and evaluation, adapted to fit a software engineering context.

The development process began with identifying the needs of the target users, namely applicants and companies and translating those needs into functional and

non-functional system requirements. Based on these requirements, a system architecture was designed, followed by the implementation of the mobile application and its backend services. The final stage involved testing and evaluating the implemented system to determine whether it met the defined objectives and research questions.

The methodology emphasizes iterative development. Features were implemented incrementally, tested and refined based on observed behaviour and system constraints. This approach allowed early identification of design issues, particularly related to authentication flow, role based access control and data synchronization.

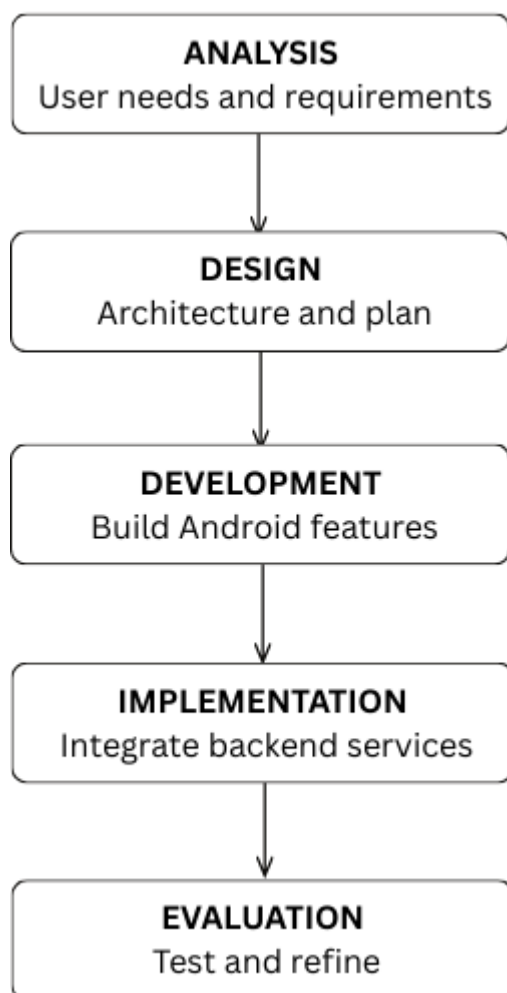


Figure 1. Simplified methodology flow (ADDIE-inspired)

The research approach also included the design and implementation of an AI-driven decision-support component aligned with the thesis objectives in Chapter 1. The AI work followed the same design-oriented methodology, where the focus was on building a practical, explainable pipeline that supports career guidance instead of developing a purely theoretical model. The AI component was developed as part of the overall system artifact, combining the Android application implementation with the AI pipeline and training resources.

The AI-related development stages were integrated into the ADDIE-inspired lifecycle. During analysis, the decision-support requirements were defined (CV-to-skill extraction, explainable role-skill matching, readiness scoring, and prioritized skill-gap recommendations). During design, the AI workflow was structured to produce transparent outputs that applicants and companies can understand, such as matched skills, missing skills, and ranked learning priorities. During development and implementation, the pipeline was integrated into the application workflow through CV intake, profile structuring, and role based presentation of AI insights. During evaluation, the AI functionality was validated through test cases to confirm consistency of extracted skills, stability of readiness scoring, and correctness of role-skill matching explanations across different CV inputs and target roles.

2.2 System Architecture and Technology

The system architecture and technology stack for this thesis were selected to support secure authentication, scalable data management, and maintainable application structure in accordance with the objectives of the study. Since the proposed system is a mobile-based career guidance application, Android was chosen as the target platform due to its widespread adoption and strong support for modern mobile application development. Kotlin was selected as the primary programming language as it is officially supported by Google and provides enhanced safety, readability and compatibility with Android development tools.

The Android application is structured using the Model-View-ViewModel (MVVM) architectural pattern. MVVM enables a clear separation between the user

interface layer, business logic and data management components. This separation is particularly important for applications that require role based access control, authentication workflows and dynamic user interface updates driven by cloud-based data.

Firebase Authentication [25] was selected as the primary service for managing user verification and login processes. Firebase Authentication provides a secure identity management solution and supports email and password-based authentication. User credentials are securely processed by the Firebase backend service, and sensitive information such as passwords is not accessible to the application or system administrators.

Cloud Firestore was chosen as the primary backend database for storing and managing application data. Firestore is a cloud-based NoSQL database that supports real-time data synchronization, automatic scaling, offline data persistence, and flexible data organization using collections and documents.

For file management, Firebase Storage was selected to store user-generated content such as applicant documents and profile-related files [26]. Firebase Storage provides a scalable cloud storage solution with secure access control rules, cloud-based buckets, and resumable upload support, ensuring reliable file transfers in mobile environments.

The selected architecture and technologies provide an integrated cloud-based solution that supports secure authentication, scalable data storage, and clean application structure. These choices align with the practical focus of the thesis and demonstrate how cloud backend services can be effectively integrated into an Android-based, role based career guidance application.

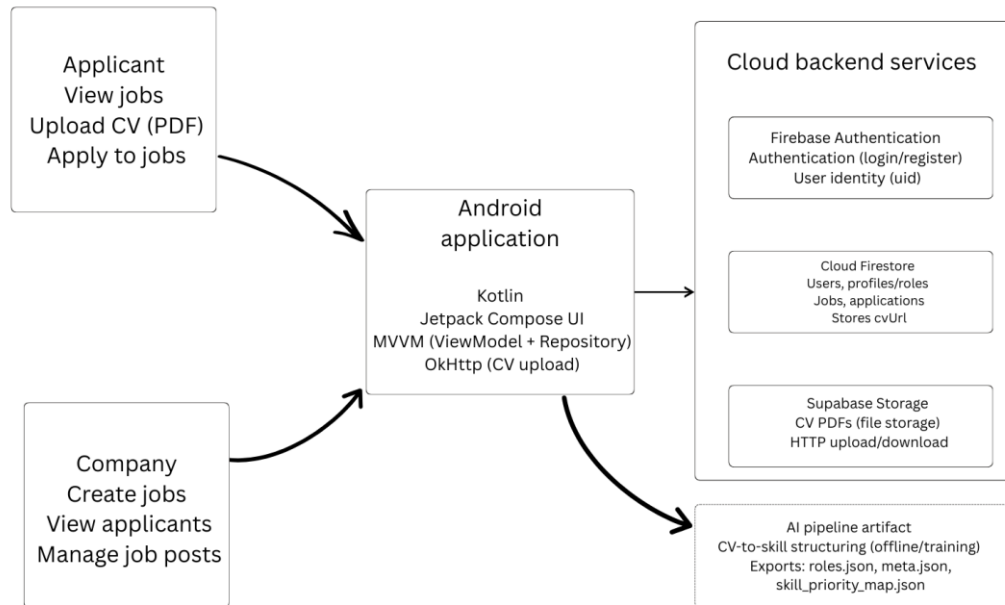


Figure 2. System architecture and technology stack

AI-driven decision-support component that is integrated into the same role based application workflow. The AI component was designed to support CV intake, profile structuring and explainable role–skill matching, while keeping the solution practical for a mobile environment. The AI-related implementation is represented by the AI pipeline artifact and its integration into the Android application.

The AI workflow follows the same architectural principles as the rest of the system. In the Android MVVM structure, CV processing and AI-related operations are handled outside the UI layer, with ViewModels coordinating user actions (such as CV upload and role selection) and repositories or service classes managing data extraction, skill structuring and score generation. This approach prevents AI logic from being tightly coupled with UI code and supports testability and future extension.

The AI pipeline was implemented using a lightweight and deployable approach suitable for student projects and mobile integration. The training and preparation steps produce structured artifacts (such as role definitions and skill priority mappings) that can be consumed by the Android app to generate transparent outputs like readiness scores, matched skills, missing skills and prioritized learning recommendations. This design supports the thesis requirement for

explainable decision support, controlled access and secure handling of sensitive CV information.

2.3 Role Based Career Guidance Approach

The career guidance application developed in this thesis follows a role based access control approach to manage user interaction and data access in a structured and secure manner. Role based access control (RBAC) [27] provides a systematic way to assign permissions based on predefined roles rather than individual users, thereby simplifying authorization management and improving system security.

In the context of this application, RBAC is applied at the application and backend service level by associating each authenticated user with a specific role. Two primary roles are defined in accordance with the scope of the thesis: applicant and company. Each role is granted a distinct set of permissions that determine which resources can be accessed and which actions can be performed. This approach aligns with RBAC principles where access rights are governed by roles rather than direct user-to-resource mappings.

Applicants interact with the system primarily as data owners. Their role permits them to create, read, and update their own profile information and upload personal documents relevant to career development. Access to applicant-specific data is restricted so that each applicant can only view or modify their own information, including applicant-only AI outputs such as extracted skills from their CV, recommended career-goal matches, readiness scores, prioritized missing skills and actionable learning recommendations generated for personal improvement. This separation of data access reflects RBAC concepts where permissions are selectively applied to protect user-specific resources.

Companies interact with the system as evaluators of applicant information. Their role allows read-only access to applicant profiles for evaluation purposes while preventing any modification of applicant data. This restriction is enforced through backend access rules and role validation mechanisms, ensuring that companies cannot access unauthorized resources or perform actions outside their assigned

role. Such separation of responsibilities is a core principle of RBAC systems and helps reduce the risk of data misuse.

By implementing role based access in this manner, the system achieves a clear separation of concerns between different user groups. The role based design simplifies backend authorization logic, reduces security risks, and improves maintainability. The RBAC approach supports future extensibility, as new roles and permission sets can be introduced without fundamental changes to the system architecture, consistent with established RBAC models, including the ability to introduce new AI-related permissions like controlling access to advanced analytics, recommendation features etc. as the system evolves.

Table 1. Role based permissions in the career guidance application

Role	Allowed actions	Restricted actions	Data Scope
Applicant	Register/Login Update own profile details Upload CV (PDF) to storage and save cvUrl View job posts Apply to jobs View own applications	Create or manage job posts View other applicants' profiles/applications Modify company job posts Any administrative actions	Own user profile and cvUrl Own applications only Read-only access to published job posts
Company	Register/login Create and manage own job posts View applicants who applied to own job posts Review applicant CV link (cvUrl) in read-only mode	Upload CV as an applicant Apply to jobs Modify applicant profile data Access applicant data outside applications to own jobs Any administrative actions	Own company profile and own job posts Read-only access to applicant application records for own jobs Read-only access to applicant cvUrl for own jobs

2.4 Authentication and Access Control Strategy

Authentication and access control form a central part of the system design, as the application handles personal profile data and provides different functionality for applicants and companies. The purpose of this section is to describe how user identity is verified and how access to system resources is controlled in a secure and structured manner.

The application uses Firebase Authentication as the identity management service. Firebase Authentication provides a centralized and secure mechanism

for verifying user credentials and managing login sessions. User authentication is performed by collecting credentials from the client application, such as an email address and password or a federated identity token and submitting them to the Firebase Authentication SDK [25]. These credentials are then verified by Firebase backend services, which return an authentication result to the application upon successful verification.

By using Firebase Authentication, sensitive operations such as password handling, credential verification, and session token generation are managed by the Firebase platform rather than being implemented manually within the application. This improves overall system security, as user passwords are never directly accessible to the application or administrators, and authentication tokens are securely issued and managed by the backend service.

After successful authentication, the application retrieves basic user profile information and uses authentication tokens provided by Firebase to identify the user in subsequent interactions. These tokens are used to control access to data stored in other Firebase services, including the cloud database. Authorization within the system is role based and depends on the role assigned to the authenticated user, which is stored as part of the user profile data.

Access control is enforced using a layered approach. At the application level, role based navigation ensures that users only see screens and features relevant to their assigned role. At the backend level, access rules restrict read and write operations based on authentication state and role information. This backend enforcement ensures that unauthorized access is prevented even if a user attempts to bypass the application interface or interact directly with backend services.

This authentication and access control strategy aligns with the objectives of the thesis by providing a secure, scalable, and maintainable solution for managing user identity and role based interaction within a mobile-based career guidance application. The use of a cloud-based authentication service [25] simplifies implementation while maintaining strong security guarantees and supporting future system extensibility.

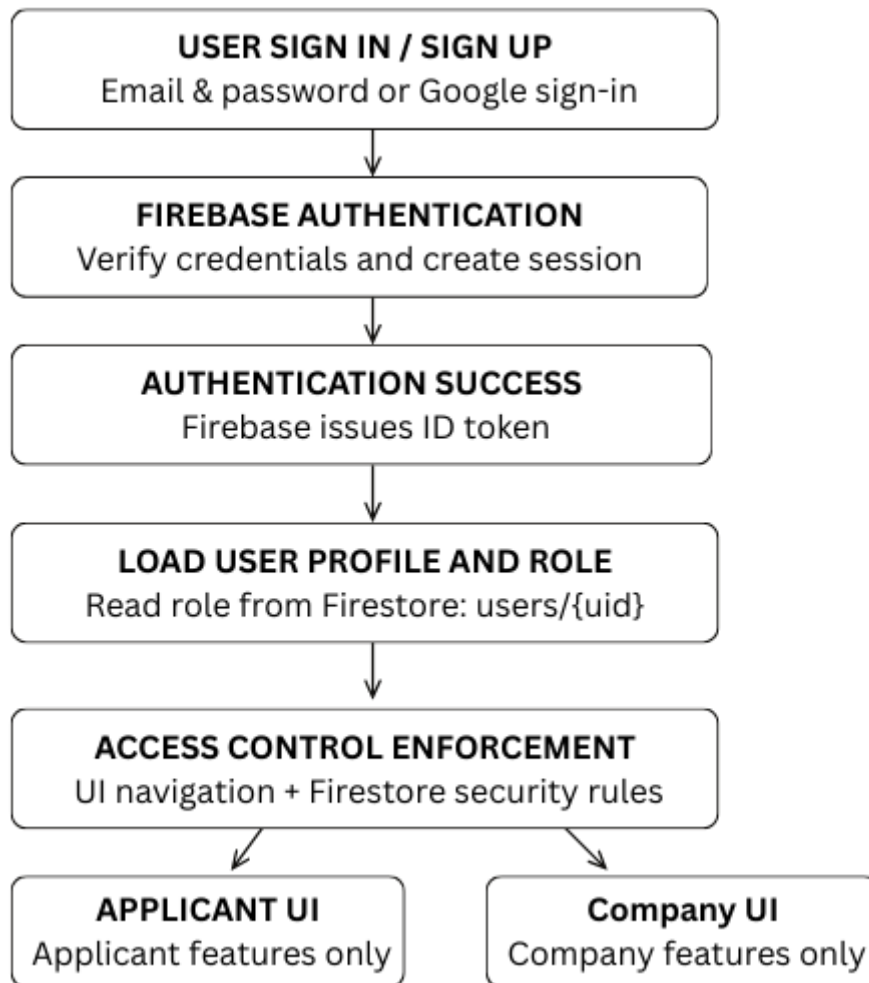


Figure 3. Authentication and role based access control flow

2.5 Cloud Based Backend Design

The backend of the career guidance application is designed using a cloud-based Backend-as-a-Service (BaaS) approach [28]. This design choice reduces the complexity associated with traditional server-based backend development, such as server hosting, database administration, API management and manual security configuration. Firebase provides an integrated cloud platform that offers real-time databases, authentication services, storage solutions and backend support for Android applications, making it suitable for scalable mobile application development.

Firebase functions as a cloud backend that enables direct communication between the mobile application and backend services over a secure connection.

It allows structured data to be stored, synchronized and retrieved efficiently without requiring explicit server-side code. Real-time data synchronization ensures that updates made by users are immediately reflected in the application, improving responsiveness and overall user experience.

In this system, Firebase Authentication [25][28] is used to manage user identity and session handling, while a cloud-hosted NoSQL database is used to store structured application data such as user profiles and role information. The database supports real-time synchronization and offline access, allowing the application to continue functioning smoothly even under unstable network conditions. Access to backend resources is controlled using authentication-aware security rules, ensuring that only authorized users can read or write data.

File related data, such as applicant documents, is handled through a cloud storage service designed for scalability, security and network resilience. Cloud storage enables users to upload files directly from the mobile application without the need for an intermediate server. Security rules are applied to restrict file access based on authentication state and user role, while resumable uploads ensure reliability in mobile network environments.

The cloud-based backend design [25] supports a clear separation of responsibilities within the system. Structured data is managed through the cloud database, while binary files are stored separately in cloud storage with references maintained in the database. This separation improves system performance, enhances maintainability, and aligns with modern mobile backend design practices. Overall, the selected backend architecture demonstrates how cloud services can be effectively used to build a secure, scalable and maintainable backend for an Android-based, role based career guidance application.

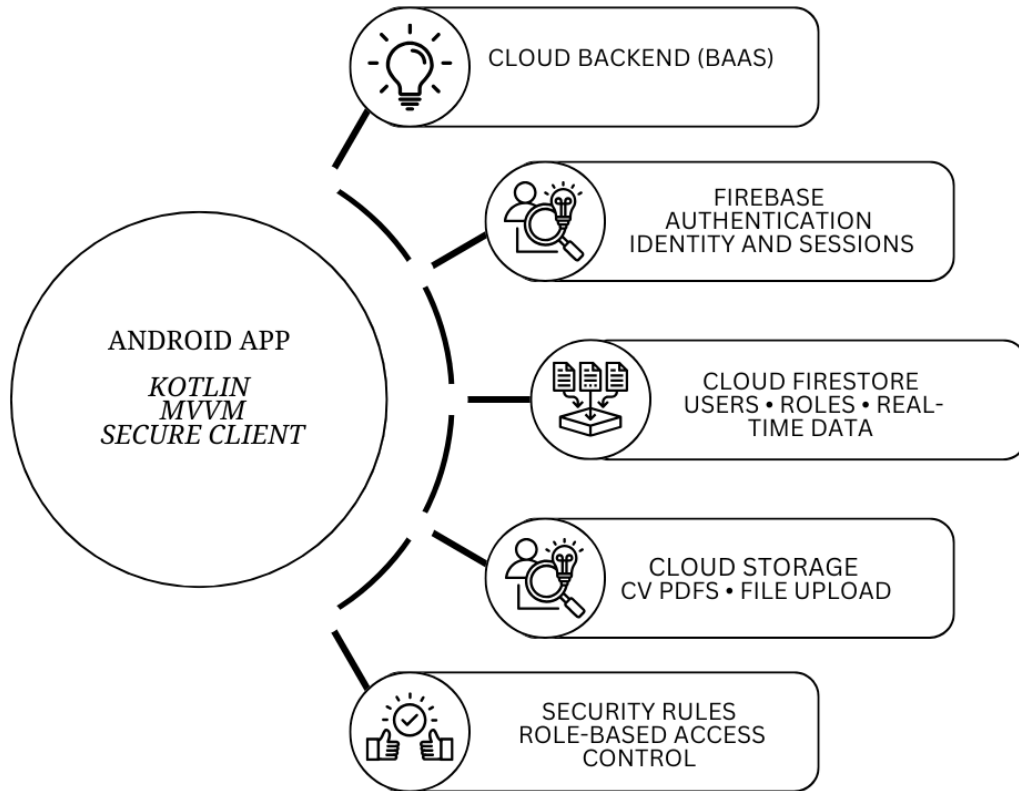


Figure 4. Cloud-based backend (BaaS) design using Firebase services

2.6 Data Collection

Although this thesis is primarily focused on system design and implementation, data collection is required to evaluate the functionality, usability and correctness of the developed application and the integrated AI-driven decision support workflow for CV-to-skill extraction, role-skill matching, readiness scoring and skill-gap recommendation output. In this study, data collection is aligned with structured usability evaluation practices [29], where interaction data is gathered during predefined testing sessions to assess system behaviour and user interaction outcomes.

The purpose of data collection is to observe and analyse how the application responds to user actions under controlled test scenarios. These scenarios are designed to simulate realistic interactions for both applicant and company roles, such as authentication, profile management, role based navigation, and file upload operations and AI-related interactions such as CV intake, text extraction,

structured skill detection, role recommendation display and learning plan generation based on detected skill gaps. Data collected during these sessions supports the evaluation of system functionality, access control enforcement, and overall interaction flow.

Each testing session follows a structured procedure. Initially, predefined test tasks are prepared to guide the evaluation process. These tasks define the actions to be performed by the tester, ensuring consistency across forty test sessions. AI-focused tasks such as uploading different CV PDFs, verifying extracted skills against expected keywords, checking stability of readiness scoring for selected roles and validating that prioritized missing skills and recommendations are generated consistently. During task execution, the system's response is monitored in real time, and relevant operational data is recorded. This approach is consistent with usability testing practices where observation and task-based interaction form the core of data collection.

The primary data collected during testing includes application behaviour during task execution, authentication and authorization [26] state transitions, database updates in Firestore and file upload outcomes in cloud storage, AI pipeline outputs such as extracted skill lists, matched and missing 1skills for target roles, readiness scores, ranked skill-gap priorities and the generated learning plan content displayed to the user. System-generated logs and error messages are collected to identify failures, inconsistencies or unexpected behaviour. These data sources provide objective evidence of whether the system meets its functional and security requirements.

Instrumentation refers to the tools and methods used to observe and record system behaviour during testing. Android Studio development tools, including Logcat, were used to monitor client-side execution and state changes, CV parsing steps, skill extraction flow, scoring calculations and AI output rendering in the UI. Firebase console logs and Firestore document inspection were used to verify backend operations and access control enforcement. These tools enabled direct observation of system events and supported detailed analysis of interaction outcomes.

Twenty-Five controlled test accounts were created for both applicant and company roles. This allowed consistent testing of role based features and access restrictions across forty sessions. By structuring data collection around task-based interaction and systematic observation, the evaluation process provides a clear and practical assessment of the implemented career guidance application in line with the objectives of the thesis.

Table 2. Instrumentation tools used during implementation and testing

Tool	What is observed	Purpose in this thesis
Android Studio (debugger)	UI navigation, state changes, crashes, stack traces	Debug app behaviour and identify implementation issues
Logcat	Runtime logs, exceptions, API errors, activity lifecycle events	Trace failures, verify app flow, and validate error handling
Firebase Console (Authentication)	Sign-in/sign-up events, user accounts, provider status (email/Google)	Verify authentication flow and session handling
Firebase Console (Cloud Firestore)	Documents, fields, read/write operations, data consistency	Confirm data storage for users/roles, jobs, applications, and cvUrl references
Supabase Storage dashboard/logs	File uploads, file paths, public URLs, access behavior	Validate CV PDF upload, stored URL generation, and document access via cvUrl

2.7 Experimental Design and Process

The system evaluation followed a scenario-based experimental design, where testing was organized around predefined interaction scenarios representing realistic user workflows [30]. Scenarios were understood as ordered sequences of interactions between a user and the system, describing how specific tasks are performed to achieve outcomes.

The experimental procedure was designed to validate whether the implemented application behaves according to its requirements and research objectives. Rather than treating testing as a final step, the evaluation process was integrated into the development workflow, allowing scenarios defined during analysis and

design to be reused for testing purposes [30]. This reuse of development artifacts supports consistency between requirements, implementation, and validation.

Each test scenario corresponds to a specific use case involving one of the two system roles: applicant or company. The evaluation was carried out in structured phases, with each phase focusing on a distinct group of system functionalities.

First, authentication scenarios were tested to verify correct registration and login behaviour using supported authentication methods. These scenarios confirmed that the system correctly transitions between authentication states and restricts access when authentication fails.

Second, role based scenarios were executed to validate access control behaviour. By signing in under different roles, the evaluation verified that role-specific navigation and permissions were enforced correctly, and that users could only access functionality associated with their assigned role [30]. These scenarios helped uncover potential ambiguities or omissions in role based authorization logic.

Third, profile management scenarios were tested by creating, viewing, and updating user profiles. The expected system behaviour was verified by observing database state changes and ensuring that data updates were correctly reflected in the application.

Fourth, file management scenarios were executed to test document upload functionality, including successful uploads and correct association of stored files with user profiles.

Fifth, AI-driven decision support scenarios were tested to validate the CV-to-skill extraction and explainable role skill matching workflow. These scenarios included uploading different CV PDF files, verifying that the system correctly extracts readable text and identifies relevant skills from the document, and confirming that the generated role recommendations, readiness scores, matched skills and missing skill gaps are consistent with the input content and the target role requirements. The evaluation also checked that prioritized learning recommendations are produced in a stable and interpretable order, and that AI-

generated outputs are displayed according to role based access rules, ensuring applicants can view their personal guidance results while company users only access evaluation relevant information when permitted.

Along with successful interaction scenarios, negative scenarios were intentionally included in the experimental design. These scenarios tested how the system behaves under error conditions, such as incorrect login credentials, missing role assignments, denied database operations due to security rules, and failed uploads caused by network interruptions.

By organizing system evaluation around scenario-based interaction sequences [30], the experimental design provides a structured and systematic approach to testing. This method supports traceability between requirements, implementation, and validation outcomes and aligns with the practical, implementation-focused objectives of the thesis.

Table 3. Scenario-based test phases and procedure

Phase	Role	What is tested	Expected outcome
Authentication	Applicant + Company	sign up, sign in, sign out	user is authenticated and session is created/ended correctly
Role based access	Applicant + Company	role detection after login and screen routing	applicant sees applicant features only; company sees company features only
Profile management	Applicant	create/update own profile and view own data	profile is stored and only the owner can update it
Document handling	Applicant	upload document (CV) and store reference/URL	CV upload succeeds to Supabase Storage, and CVurl is saved in Firestore
Company evaluation access	Company	view applicant information for	company can view allowed data but cannot edit applicant data

		evaluation (read-only)	
Negative scenarios	Both	invalid login, unauthorized access attempts, missing permissions	access is denied and app stays secure

2.8 Data Analysis

The evaluation of the developed system follows a requirements-based testing approach, where system behaviour is assessed against predefined functional and security requirements [31]. The purpose of defining explicit measures is to ensure that the implementation satisfies the stated objectives and that all core requirements are validated through systematic testing.

Functional correctness was measured by verifying whether the application fulfils its defined requirements through predefined test cases. These test cases were derived directly from the functional requirements of the system, including user registration, authentication, role identification, role based navigation, profile creation and updates, document upload functionality, the AI-driven user workflow where an applicant uploads a CV, the system performs CV-to-skill extraction, generates explainable role recommendations and role-skill matching results, produces readiness scoring, and displays prioritized skill-gap recommendations with learning guidance. Each test case specified an initial system state, required inputs, and expected outcomes, allowing objective verification of correct behaviour.

Security enforcement was measured by validating access control requirements using backend security rules. The evaluation verified that applicants could read and modify only their own profile data and that company users were restricted to read-only access for applicant profiles. Successful enforcement was confirmed when unauthorized actions resulted in access denial and no unintended data modifications occurred. This measure supports early detection of authorization defects, which is a key objective of requirements-based testing [31].

Usability related behaviour was evaluated in relation to requirement compliance rather than subjective user satisfaction. Measures focused on whether defined user workflows could be completed as intended, including logical navigation after login, clear role-specific screen availability, and appropriate system feedback in error conditions. These observations ensured that the implemented behaviour matched the expected outcomes defined by the requirements.

Data analysis in this thesis is descriptive and requirement oriented. Test results were analysed by comparing observed system behaviour with expected outcomes defined in the test cases. Test completion criteria were satisfied when all critical requirements were successfully validated, and no blocking defects remained. Failed test cases and unexpected behaviour were documented and analysed to identify implementation limitations or areas for improvement.

By aligning measures and analysis directly with system requirements, this evaluation approach supports early defect detection, ensures adequate test coverage of core functionality, and provides clear traceability between requirements, implementation, and testing outcomes. This approach is consistent with applied software engineering research, where evaluation focuses on confirming that the implemented system meets its intended requirements under expected operating conditions.

Table 4. Evaluation measures and pass criteria

Measure	How verified	Pass criteria
Functional correctness	Scenario tests of core workflows	All core workflows complete without errors
Security enforcement (RBAC)	Role tests + unauthorized action attempts	Unauthorized read/write is blocked by rules
Data consistency and synchronization	Firestore updates observed across sessions/devices	Updates reflect correctly and remain consistent
Usability-related behaviour	Manual walkthrough and observation of navigation	Users can complete tasks with clear steps

2.9 Trustworthiness and Validity Factors

Trustworthiness and validity are important considerations in evaluating the reliability of the results obtained in this thesis. Validity cannot be assessed only at the analysis stage but must be considered throughout the design, implementation, and evaluation phases of the study. In this section, trustworthiness and validity are discussed in relation to the evaluation methods used.

Construct validity refers to whether the evaluation measures accurately represent what is intended to be measured according to the research questions [32]. In this thesis, construct validity is supported by aligning test scenarios directly with functional and security requirements derived from the system objectives. The test cases focus on concrete system behaviours such as authentication, role based access control, profile management, and file upload functionality, ensuring that the measured outcomes correspond to the intended constructs.

Internal validity is concerned with causal relationships and whether observed effects can be attributed to the implemented system rather than external factors. Since this thesis does not investigate causal relationships between independent variables, internal validity threats are limited. To reduce potential influence from external factors, testing was conducted in a controlled environment using the same device, test accounts, and configuration settings across repeated test runs.

External validity addresses the extent to which the findings can be generalized beyond the specific context of the study [32]. The evaluation results are limited to the tested Android environment and a small set of predefined scenarios. While statistical generalization is not intended, the study supports analytical generalization by providing detailed descriptions of the system context, architecture, and evaluation procedure, allowing readers to assess the relevance of the findings to similar applications.

Trustworthiness concerns whether the evaluation process would produce the same results if repeated under similar conditions. Trustworthiness was improved by executing the same test scenarios multiple times using identical test data and

workflows. System behaviour was considered reliable when repeated executions produced consistent outcomes without unexpected variation.

The evaluation does not include a large-scale user study, and usability observations are primarily based on developer-led testing rather than multiple independent participants. Additionally, the scope of testing does not cover long-term usage or high-load conditions. These threats are mitigated by clearly documenting the evaluation process and identifying these limitations as areas for future improvement, which is consistent with good study design practices.

Table 5. Validity and trustworthiness factors

Aspect	Factor	How it was addressed
Construct validity	Do the measures represent the intended objectives?	Measures mapped to requirements and research questions
Internal validity	Could other factors affect the results?	Same scenarios and steps repeated; issues verified using logs
External validity	How well do results generalize?	Evaluation focused on prototype in a controlled setting; scope stated
Trustworthiness	Would results be consistent if repeated?	Test steps documented; outcomes cross-checked with Firebase/Logcat

2.10 Data Protection and Ethical Factors

Data protection and ethical factors are important in this thesis because the application processes personal information related to applicants and company users. Mobile applications can raise privacy and security risks due to the variety of personal data involved, the mobile ecosystem, and the use of external services, which makes compliance with data protection principles more challenging in practice [33]. Even though this work is a student project and not deployed as a public commercial service, responsible handling of user data is still required.

The system design follows the idea of data protection by design and by default by considering privacy and security during the development lifecycle rather than only at the end of implementation. The application was designed to minimize unnecessary data collection, and only information required for core profile management and system functionality is stored [33]. This supports the principle of data minimisation by limiting collected data to what is necessary for the defined purpose.

Access to personal data is restricted through authentication and backend access control rules to support integrity and confidentiality of stored information. Role based access control ensures that applicants can manage only their own profile data, while company users have access only to permitted viewing functionality. Authentication credentials and password security are handled by Firebase Authentication, meaning passwords are not stored or accessible within the application itself, which reduces exposure of sensitive credentials [25][27].

The use of cloud backend services was treated as a privacy decision point, because cloud storage and third-party services can introduce additional risk sources if not configured securely. To mitigate these risks, backend security rules were used to control database and storage access, and test accounts were configured with clear separation between applicant and company roles.

During development and evaluation, only non-sensitive sample data was used. No real applicant data from external individuals was collected, and no personal data was shared outside the testing environment. These measures reduce ethical risks related to confidentiality and avoid unintended exposure of personal information within the scope of this thesis.

Ethical and data protection considerations apply to the AI-driven decision-support pipeline because it processes CV documents and generates outputs that may influence career related decisions. To reduce privacy risks, the AI workflow was designed to use only the minimum information required to produce the intended functionality, where the CV is processed for text extraction and skill identification rather than for unnecessary personal attribute profiling. The AI pipeline implementation artifacts support a practical approach where training and

prioritization logic are based on role and skill definitions and synthetic or controlled test inputs rather than real external applicant datasets, which reduces risks of collecting sensitive personal data. The integration in the Android application was implemented with controlled access principles, ensuring that AI-generated results such as extracted skill lists, role-skill matching explanations, readiness scores and skill gap recommendations are visible only to authorized users according to role based access control rules. The system design emphasizes explainable outputs and human oversight, so recommendations are presented with clear reasons (matched and missing skills) to support responsible use of AI guidance without making automated decisions on behalf of the user.

2.11 Limitations of Methodology

The methodology adopted in this thesis has several limitations that should be considered when interpreting the results. The evaluation of the system was conducted using controlled test accounts and developer driven test scenarios rather than a large and diverse group of real end users, which limits the representativeness of the findings. This approach resembles convenience sampling, where subjects or test conditions are selected primarily due to accessibility rather than statistical representativeness of a broader population.

Because the system was evaluated primarily by the developer, the results reflect a specific sub-type of user behaviour and technical interaction rather than the full diversity of applicant and company usage patterns that may occur in real world deployment. Convenience based evaluation limits external validity and restricts the extent to which results can be generalized beyond the tested context [34].

The evaluation design focused mainly on functional correctness, role based access control, and backend security enforcement. While this is appropriate for an implementation-focused software engineering thesis [34], it limits insight into broader usability, accessibility and long-term user behaviour, which would require structured user studies involving participants with varied backgrounds and experience levels.

Another limitation is that the evaluation was performed in a controlled development environment with stable configurations. Factors such as large-scale concurrent usage, heterogeneous devices and varying network conditions were not systematically tested. As a result, system behaviour under high load or in less predictable real-world environments remains outside the scope of this study.

Next limitation is that the application depends on external cloud services for authentication, database operations and storage. Any limitations, outages or policy changes in these services can affect system behaviour independently of the application's internal design. These dependencies are inherent constraints of cloud-based application development and are acknowledged as part of the methodological boundaries of this thesis.

The limitation of the Artificial Intelligence implication concerns the AI-driven decision support pipeline used for CV-to-skill extraction, role-skill matching, readiness scoring and skill gap recommendations. The AI component was validated mainly through controlled CV inputs and predefined role datasets rather than through large scale evaluation with diverse real world CV formats, writing styles and language variations that may affect the robustness of extraction and matching performance in practical deployment. The AI pipeline logic and training resources rely on structured role skill definitions and developer prepared or synthetic testing conditions which means the evaluation confirms functional consistency and explainability within the implemented system context but does not provide statistically grounded accuracy measures against externally labelled ground truth data. As a result, potential limitations such as sensitivity to ambiguous CV wording, incomplete CV content, differences in terminology and the influence of the role dataset design on recommendations remain outside the validated scope of this study.

3 SYSTEM DESIGN AND IMPLEMENTATION

3.1 System Design and Development Structure

The system for Career Guidance App was designed as an Android mobile application that demonstrated how role based career guidance, job discovery, and application workflows could be executed within a single logical architecture. Clarity, traceability and technical sufficiency were given top priority in the design philosophy, which made sure that every essential user flow outlined in the thesis scope was directly implemented in the codebase. All android application mechanisms like authentication, role separation (applicant vs company), job posting, job browsing, application submission, profile maintenance and CV handling were used in a manner that remained relevant, reusable and verifiable. The AI-driven decision support part described in the thesis was implemented as an explainable CV-to-skill and role-skill matching workflow inside the same architecture, where extracted CV signals like skills and experience feed role recommendations, readiness scoring and prioritized skill-gap outputs that are visible to the user as transparent.

Using Firebase Authentication and Cloud Firestore as the main cloud backend services and Supabase Storage as a document storage provider for applicant CV files, the platform used a practical client-driven strategy. Deterministic UI navigation rules, explicit Firestore document models, and strongly typed Kotlin modules were used in place of a custom backend server early in the prototype stage. For instance, a straightforward role field kept in Firestore (`users/{uid}.role`) was used to provide role-driven behaviour, which was then enforced throughout the user interface by limiting the actions that were accessible on the job screens and home dashboard. The AI component was integrated without a dedicated inference server by packaging the AI outputs as versioned static assets used by the Android app, where the AI pipeline produces model derived skill priorities (`skill_priority_map.json`) from training code and exports them into the mobile project so that skill-gap ranking and readiness scoring can be computed in a

deterministic, inspectable way on the client while still reflecting a trained model's learned weighting.

The app followed a three-tier logical architecture [35], implemented in a single Android application module:

1. Presentation Layer

The client-side interfaces (Jetpack Compose [35] screens under `app/src/main/java/com/example/careerguidance/ui/`) handled user interaction, navigation, job rendering, application forms, and profile editing. Screens were mapped directly to functional requirements such as login, registration, job browsing, job creation, applicant listing, and profile viewing/editing.

2. Application Layer

Application flow is coordinated through Compose Navigation defined in `MainActivity.kt` and state management in ViewModels [36] for authentication-related screens (`LoginViewModel` and `SignupViewModel`). Side-effects, such as Firestore writes and CV uploads, are executed from UI actions using the Firebase SDK and `OkHttp`.

3. Data and Security Layer

Firebase Authentication ensures each action can be linked to a signed-in user. Firestore persists with user profiles and job/application data [22][25]. Supabase Storage persists in uploading CV PDFs and returns a URL that is stored in Firestore so that applications and company review screens can access it.

A single-app architecture was chosen to keep the thesis prototype contained and demonstrable without the overhead of deploying and maintaining a separate API server. Development was done incrementally: authentication and navigation were established first, followed by role persistence at signup, job posting and job listing and finally applicant CV uploading and application submission with per-job applicant review. After these core android workflows were stable, the AI-driven guidance layer was connected end-to-end by parsing uploaded CV PDFs into text, extracting and normalizing skill signals, applying role-skill matching logic,

and generating explainable outputs (recommended roles, readiness score, and prioritized missing skills/learning priorities), with the resulting recommendation/plan data stored and retrievable through Firestore so that the AI-related outputs remain traceable and verifiable.

3.2 Complete System Architecture

The system is implemented as an Android application written in Kotlin, using Jetpack Compose for interface rendering and managed cloud services for backend responsibilities. The architecture supports role based access and a clear workflow for job marketplace interactions: companies publish job posts, applicants browse jobs, applicants apply (with a CV reference), and companies review applicants. The architecture supports an AI-driven decision support workflow: applicants upload a CV (PDF), the document is processed into structured profile signals (skills and experience), and the system produces explainable role-skill matching outputs such as a readiness score and prioritized skill-gap recommendations.

The architecture can be viewed as three cooperating layers inside one repository: a Compose client layer, an orchestration layer (navigation and state handling), and a cloud service layer (authentication, database and storage). This separation helped keeping UI responsibilities focused on interaction and rendering while the persistent state remained in cloud services. The same layered separation also supported the AI-driven decision support workflow that allowed fast repetition and traceable integration because each stage of the AI pipeline (CV PDF upload → text extraction → skill signal structuring → role-skill matching → readiness scoring → prioritized skill-gap and learning recommendations) was executed as a united flow within the Android application and backed by inspectable cloud-stored outputs.

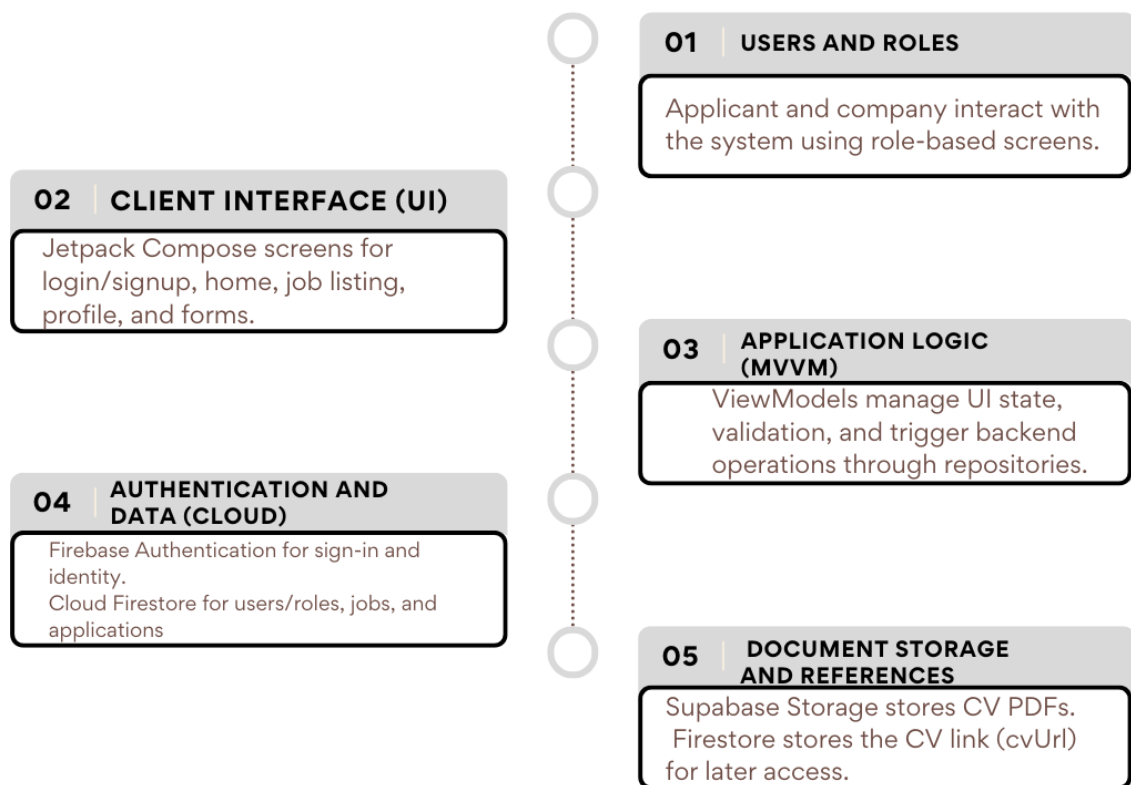


Figure 5. System architecture flow overview

3.2.1 Design Logic

The following practical constraints and project goals influenced the architecture:

- **Single project, mobile first delivery**

A unified Android project makes prototype contained and directly demonstrable, allowing the complete workflow to be inspected from Jetpack Compose screens through navigation or state handling to cloud persistence. Core career guidance flows (authentication, role based dashboards, job browsing or posting, application submission, profile maintenance and CV intake) can be executed and verified in one environment, supporting the applied, implementation-focused methodology and ensuring traceability between scope requirements and the delivered code.

- **Managed backend services for rapid prototyping**

The system requires user authentication, structured records (profiles/jobs/applications) and CV file persistence. Firebase Authentication and Cloud Firestore provide practical, scalable services for identity and database operations without maintaining a custom server, which aligns with the project's client-driven design and the need for reproducible evaluation. Supabase Storage is used for CV PDF uploads to keep large files out of Firestore while still enabling consistent access through stored URLs, and the same backend structure supports storing and retrieving AI-related outputs like extracted skills, readiness scores and recommendations.

- **Role based behaviour with controlled access to AI insights**

The architecture encodes role as a persistent field and uses it consistently to enforce role-specific workflows across UI navigation and role based actions, reducing the need for complex client side authorization logic while preserving clear separation between applicant and company capabilities. For example, companies access job creation and applicant review flows, while applicants access CV upload, applications and AI-guided skill-gap feedback.

3.2.2 High Level Architecture

Overall system is composed of the following layers:

Layer 1: User Interface (Jetpack Compose UI)

The layer is responsible for:

- Collecting credentials and registration details
- Presenting role-specific actions on the home screen
- Rendering job lists and job cards
- Enabling job posting for companies
- Enabling application submission for applicants
- Showing applicant lists for a selected job

- Displaying AI-driven decision support outputs to applicants that includes explainable role-skill matching readiness score and prioritized skill-gap recommendations with learning guidance

Layer 2: Application Orchestration (Navigation and ViewModels)

The orchestration layer includes:

- Route definitions and navigation control in MainActivity.kt
- Authentication state handling (sign-in status affecting navigation)
- ViewModels that expose UI state for login and signup flows
- Controlled side-effects such as Firestore writes and CV upload actions
- AI workflow coordination that includes CV intake handling which is PDF to text extraction, skill signal structuring and execution of role-skill matching to generate readiness scoring and ranked gap outputs consumed by the UI

Layer 3: Cloud Services (Firebase Auth, Firestore, Supabase Storage)

The cloud layer provides:

- Identify and session tokens via Firebase Authentication
- Persistent structured data via Firestore (users/jobs/applications)
- CV file storage via Supabase Storage (upload + URL reference)
- Consistent AI-related structured outputs (extracted skill signals, readiness scores and skill-gap recommendations) linked to user profiles and applications

Layer 4: AI Pipeline

The AI pipeline supports the app by:

- Training and deriving the decision support artefacts and mappings used for skill prioritization and matching logic
- Exporting the produced artefacts for integration into the Android project so the app can generate consistent, explainable decision support results

3.2.3 Workflow for Request Response

Although this is a mobile application rather than a server-rendered web platform, user actions still follow predictable request/response patterns through cloud SDK calls.

Example Workflow A: Company job posting

Step 1: UI input

- JobCreateScreen collects job fields and reads company identity from `users/{uid}`.

Step 2: Firestore write

- A new document is added to jobs with `companyId`, `companyName`, and job details.

Step 3: UI refresh

- JobsListingScreen reflects the new job through Firestore snapshot updates or re-fetch logic.

Example Workflow B: Applicant applying to a job

Step 1: UI action

- Applicant taps Apply on a job card in JobsListingScreen.

Step 2: Precondition check

- The app checks whether `users/{uid}.cvUrl` exists before allowing application submission.

Step 3: Firestore check

- The application is saved under `jobs/{jobId}/applications/{applicantUid}` with applicant identity fields and `cvUrl`.

Step 4: UI status update

- The job card is updated to show that the applicant has already applied (by detecting the existing application document).

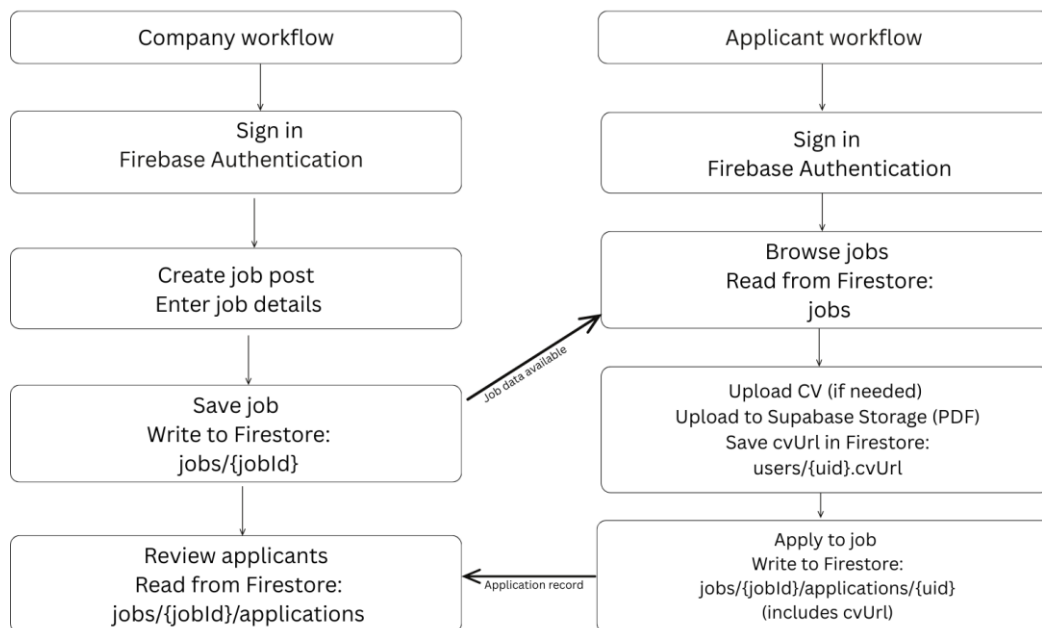


Figure 6. Request-response workflow for job posting and job application

Example Workflow C: AI-driven CV processing and decision-support outputs

Step 1: UI action

- Applicant uploads a CV PDF from the profile screen, triggering the CV intake flow.

Step 2: Storage write

- The CV file is uploaded to Supabase Storage and a `cvUrl` reference is obtained.

Step 3: Firestore update

- The cvUrl is stored under users/{uid} so the CV can be referenced consistently across application and review screens.

Step 4: AI processing and structuring

- The application extracts CV text, normalizes the content into structured skill signals and executes the role-skill matching logic using AI pipeline.

Step 5: Firestore persistence of AI outputs

- The resulting explainable outputs are stored in Firestore under the user profile and application records.

Step 6: UI feedback and refresh

- The UI displays the readiness score and ranks skill gaps as actionable guidance and refreshes from Firestore to keep the decision support outputs consistent across sessions and screens.

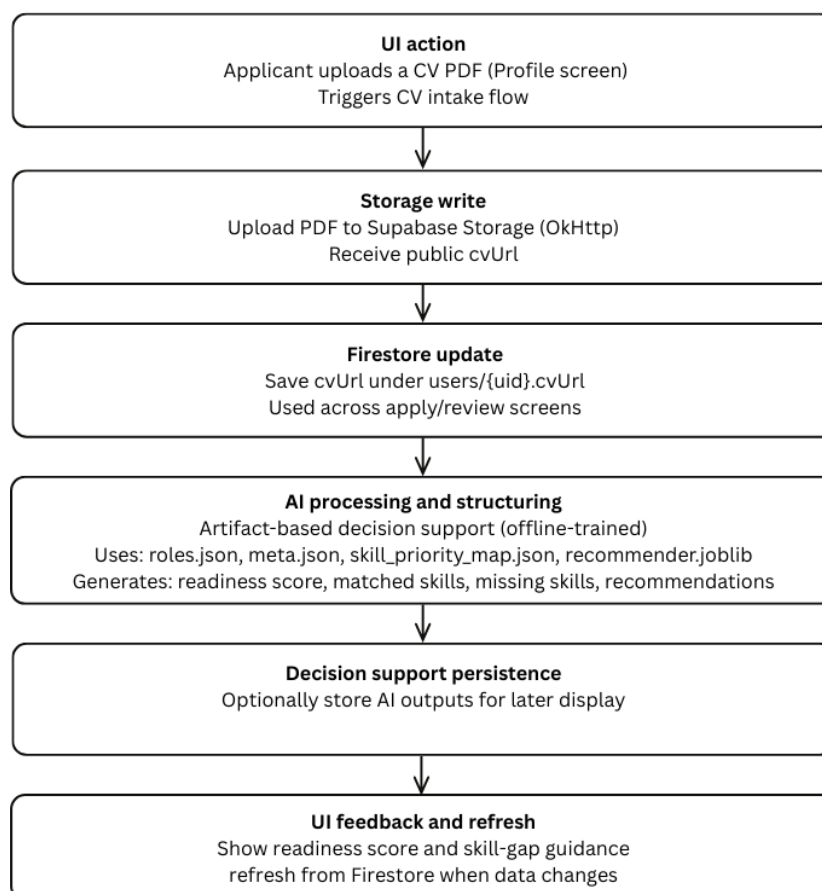


Figure 7. AI-driven CV processing and decision-support workflow

3.3 Design of Data Model and Schema

The data model is implemented using Firestore collections for structured records and Supabase Storage for PDF documents. The model is designed to support role based identity and profiles, job publishing and job application tracking and CV persistence as a separate storage concern, extracting profile signals and storing skill readiness score obtained from AI pipeline.

At a high level, the data model can be grouped into:

- Users and profile data (Firestore users collection)
- Jobs and applications (Firestore jobs collection and applications subcollections)
- CV documents (Supabase Storage objects referenced by URL)
- AI-derived profile and matching outputs (Firestore collections storing extracted skills, readiness scores, ranked skill-gap and learning recommendations)

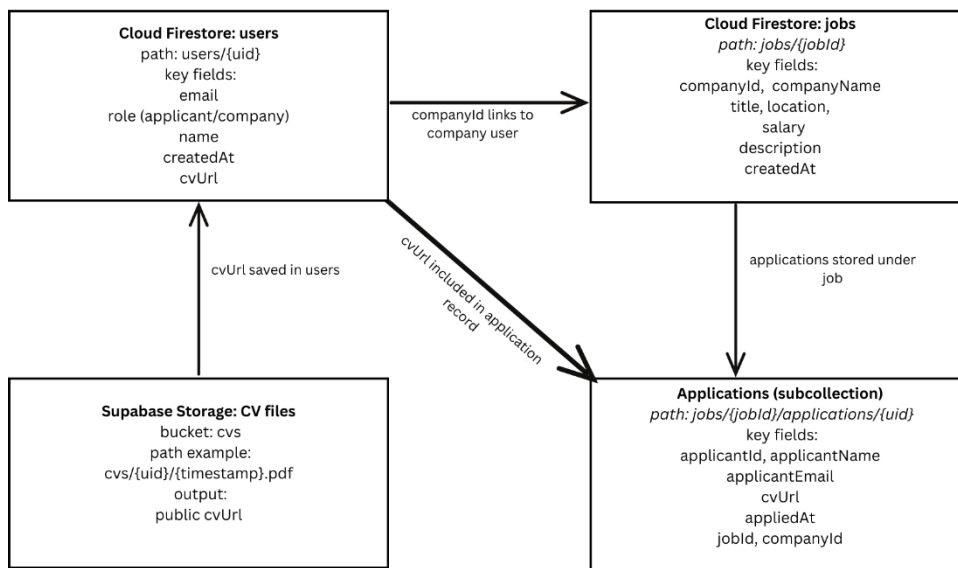


Figure 8. Firestore data model and document relationships (users, jobs, applications, and cvUrl)

3.3.1 User Profile and Identification

Firebase Authentication creates and validates identity, but application-specific profile details are stored in Firestore under `users/{uid}`. The user document includes a role field and profile fields that differ slightly depending on whether the user is an applicant or a company. The profile document also stores `cvUrl`, which is used as a prerequisite for applying to jobs and getting AI driven decision support results.

The profile design is intentionally simple: the UI reads a single user document to render the home dashboard and profile page, and it uses direct updates for individual profile fields.

Table 6. User profile fields stored in Firestore (users/{uid})

Field (users/{uid})	Description	Applies to
Email	Account email address	Both
Role	User role: applicant or company	Both
CreatedAt	Account creation time (server timestamp)	Both
Name	Display name used in profile and UI	Both
NameLocked	Prevents name changes after initial save	Both
City	User city (profile detail)	Applicant
Phone	Phone number (profile detail)	Applicant
Bio	Short description (profile detail)	Applicant
EducationLevel	Education level (profile detail)	Applicant
Major	Field of study (profile detail)	Applicant
Location	Preferred/current location (profile detail)	Applicant
GoalRole	Target role/job goal	Applicant
Cvurl	Public CV link saved after upload	Applicant

3.3.2 Job Records and Application

The job marketplace is modelled as a jobs collection with an applications subcollection per job. This structure supports the required read patterns efficiently: applicants list jobs globally, while companies list their own jobs and view applicants for a specific job.

Jobs collection fields:

- companyId (owner uid)
- CompanyName
- Title, location, salary, description
- createdAt timestamp

Table 7. Job record fields stored in Firestore (jobs/{jobId})

Field (jobs/{jobId})	Description
CompanyId	UID of the company that posted the job
CompanyName	Company name shown in job cards and details
Title	Job title
Location	Job location (city/region)
Salary	Salary text/value stored for display
Description	Job description text
CreatedAt	Job posting creation time

jobs/{jobId}/applications fields:

- applicantId (uid)
- applicantName, applicantEmail
- cvUrl
- appliedAt timestamp

Table 8. Application record fields stored in Firestore

Field	Description
jobs/{jobId}/applications/{uid}	

applicantId	UID of the applicant who applied
applicantName	Applicant name saved at the time of applying
applicantEmail	Applicant email saved at the time of applying
cvUrl	Public CV link referenced for evaluation
appliedAt	Application submission time (timestamp)
jobId	Identifier of the job that was applied to
companyId	UID of the company that owns the job post

The document id strategy for applications (using applicant uid) implicitly prevents duplicates for the same job because one applicant can only create one document with that id.

3.3.3 CV Storage References

CV files are stored as PDF objects in Supabase Storage. After upload, a URL is written back into Firestore as `users/{uid}.cvUrl` and the text from CV pdf is extracted which updates the profile of the applicant that is visible in applicant's profile. When an applicant applies for a job, the `cvUrl` is also copied into the application document, so the company can review the CV even if the applicant later uploads a newer file.

3.3.4 AI Decision Support State

AI decision support relied on a set of assets and Firestore documents which separated static definitions from user specific state. `roles.json` defined each target role and its required or optional skills, while `skill_priority_map.json` provided the learned importance weights exported by `career_ai_training/train.py` and `career_ai_training/export_android.py` and loaded in the app via `SkillPriorityRepository`. `users/{uid}` stored the current profile signals used for

matching, including goalRole, experienceYears, skillsRaw, skillsNormalized and cvUrl. goalPlanProgress stored the ongoing completion state as a fixed document with type = "progress" and completedSkills. All these records allowed the system to demonstrate a essential readiness and skill-gap summary on the recommendation and action-plan screens, preserving traceability between decision-support outputs and the stored profile state.

3.4 Core Role Based Workflow Logic

The core logic of the thesis is centered on role based workflows that control which actions are possible and how data is written. The intent is that behaviour differences are driven by explicit stored values rather than implicit heuristics, so that every outcome can be traced to a clear condition.

The implementation is distributed across:

- UI screens (HomeScreen.kt, LoginScreen.kt, SignupScreen.kt, JobsListingScreen.kt, JobCreateScreen.kt, ApplicantsListScreen.kt, ProfileScreen.kt, RecommendationsScreen.kt, ActionPlanScreen.kt, SkillImpactScreen.kt)
- Navigation (MainActivity.kt)
- ViewModels and state holders (LoginViewModel.kt, SignupViewModel.kt, ProfileViewModel.kt, RecommendationsViewModel.kt, GoalPlanViewModel.kt, ActionPlanViewModel.kt, SkillImpactViewModel.kt, LoginState.kt, SignupState.kt, ProfileState.kt, RecommendationState.kt)
- Data and AI modules (CvParser.kt, GoalPlanEngine.kt, RecommendationEngine.kt, RolesRepository.kt, RoleCatalogDataSource.kt, SkillPriorityRepository.kt, SkillNormalizer.kt, Role.kt, RoleDefinition.kt, Recommendation.kt, LearningContentRepository.kt, LearningContent.kt, SupabaseConfig.kt)
- Firebase Auth and Firestore calls are made from MainActivity.kt, screens, and ViewModels (login/signup, role persistence, job/application writes, profile reads/updates).

- Supabase Storage upload configuration is in SupabaseConfig.kt and is invoked from the CV upload flow.

```
career_ai_training/
├── .venv/
│   ├── Lib/site-packages/
│   ├── Scripts/
│   ├── .gitignore
│   └── pyvenv.cfg
├── out/
│   ├── meta.json
│   ├── recommender.joblib
│   └── skill_priority_map.json
├── export_android.py
├── requirements.txt
├── roles.json
└── train.py
```

Figure 9. AI pipeline structure

```

app/
├─ src/
│  ├─ androidTest/
│  │  └─ java/com.example.careerguidance/
│  │     └─ ExampleInstrumentedTest
│  └─ main/
│     ├─ AndroidManifest.xml
│     ├─ assets/
│     │  ├─ learning_content.json
│     │  ├─ roles.json
│     │  └─ skill_priority_map.json
│     └─ java/com.example.careerguidance/
│        ├─ MainActivity
│        ├─ data/
│        │  ├─ SupabaseConfig
│        │  ├─ cv/
│        │  │  └─ CvParser.kt
│        │  ├─ plan/
│        │  │  ├─ LearningContent
│        │  │  └─ LearningContentRepository
│        │  └─ recommendation/
│        │     ├─ GoalPlanEngine.kt
│        │     ├─ Recommendation
│        │     ├─ RecommendationEngine
│        │     ├─ Role
│        │     ├─ RoleCatalogDataSource
│        │     ├─ RoleDefinition
│        │     ├─ RolesRepository
│        │     ├─ SkillNormalizer
│        │     └─ SkillPriorityRepository.kt
│        └─ ui/
│           ├─ HomeScreen.kt
│           ├─ auth/
│           │  ├─ LoginScreen.kt
│           │  ├─ LoginState
│           │  ├─ LoginViewModel
│           │  ├─ SignupScreen.kt
│           │  ├─ SignupState
│           │  ├─ SignupViewModel
│           │  └─ UserRole
│           ├─ impact/
│           │  ├─ SkillImpactScreen.kt
│           │  └─ SkillImpactViewModel.kt
│           ├─ jobs/
│           │  ├─ ApplicantsListScreen.kt
│           │  ├─ JobCreateScreen.kt
│           │  └─ JobsListingScreen.kt
│           ├─ plan/
│           │  ├─ ActionPlanScreen.kt
│           │  └─ ActionPlanViewModel.kt
│           ├─ profile/
│           │  ├─ ProfileScreen.kt
│           │  ├─ ProfileState
│           │  └─ ProfileViewModel
│           ├─ recommendation/
│           │  ├─ GoalPlanViewModel.kt
│           │  ├─ RecommendationsScreen.kt
│           │  ├─ RecommendationState
│           │  └─ RecommendationsViewModel
│           └─ theme/
│              ├─ Color.kt
│              ├─ Theme.kt
│              └─ Type.kt
├─ res/
│  ├─ drawable/
│  ├─ mipmap-anydpi-v26/
│  ├─ mipmap-hdpi/
│  ├─ mipmap-mdpi/
│  ├─ mipmap-xhdpi/
│  ├─ mipmap-xxhdpi/
│  ├─ mipmap-xxxhdpi/
│  ├─ values/
│  └─ xml/
├─ test/
│  └─ java/com.example.careerguidance/
│     └─ ExampleUnitTest
├─ build.gradle.kts
├─ google-services.json
├─ proguard-rules.pro
└─ .gitignore

```

Figure 10. Android project package structure (implementation overview)

3.4.1 Role Resolution and Home Dashboard

After a user sign in, the app reads the user profile document from Firestore and determines the current role. HomeScreen is located at `app/src/main/java/com/example/careerguidance/ui/HomeScreen.kt` and renders role-specific actions:

- Applicant: upload CV, browse jobs, open profile, get decision support results
- Company: create job, manage jobs, open profile

3.4.2 Job Listing Rules

The JobsListingScreen uses role to select the correct query pattern:

- Applicants see job listings from all companies
- Companies see only jobs where `companyId` equals their `uid`

This prevents companies from managing jobs that do not belong to them and keeps the UI aligned with ownership.

3.4.3 Application Rules and Validation

Job applications are permitted only when:

- A user is authenticated
- The user role is applicant
- The user has uploaded a CV (`cvUrl` exists)

If these conditions are met, the application document is written to the applications subcollection. If the document already exists for that applicant id, the UI treats the job as already applied.

Table 9. Role based UI, query, and validation rules in the application

Rule area	Role	Condition	System action
Home dashboard	Applicant	role = applicant	Show applicant actions (upload CV, browse jobs)
Home dashboard	Company	role = company	Show company actions (create job, manage jobs, view applicants)
Job listing query	Applicant	role = applicant	Read jobs ordered by createdAt
Job listing query	Company	role = company	Read jobs where companyId = uid
Apply validation	Applicant	authenticated + cvUrl exists + not applied	Write jobs/{jobId}/applications/{uid}
Apply blocked	Company	role = company	Hide/disable apply action
Applicants list access	Company	Owns job (companyId = uid)	Read jobs/{jobId}/applications for review

3.5 Job Posting and Applicant Review Implementation

The job functionality workflow links company job creation, applicant job discovery, application submission with a stored cvUrl, and application review by company. Companies create new job documents from JobCreateScreen.kt, and these postings are rendered to applicants in JobsListingScreen.kt through Firestore reads and refresh logic. Applicants submit applications by writing their details and cvUrl into the job's applications records, and the UI reflects applied status by detecting an existing application entry for the same job. Companies open ApplicantsListScreen.kt to fetch and display the list of applicants per job.

3.5.1 Job Posting Flow

JobCreateScreen is available only to company users through the home dashboard route. The screen collects job details and writes a document to the jobs collection. Company identity is included through companyId and companyName.

3.5.2 Application Submission

Applicants apply using the Apply action on job cards. Application creation stores applicant identity and cvUrl so that a company can review the application without additional lookups.

3.5.3 Applicant Review for Companies

The ApplicantsListScreen loads applications from jobs/{jobId}/applications and renders each applicant entry. CV links are opened using Android intents to delegate PDF viewing to external applications.

3.6 CV Upload and Document Handling Implementation

CV upload is implemented as a separate workflow because PDFs are not stored directly inside Firestore documents. The app uses the Android document picker to select a file, uploads it to Supabase Storage with OkHttp, and stores the resulting URL in Firestore.

Key implementation components include:

- `ActivityResultContracts.OpenDocuments` for PDF selection
- Supabase upload configuration (`SupabaseConfig.kt`)
- Firestore update of `users/{uid}.cvUrl` after successful upload

3.6.1 CV Upload Procedure

The sequence is:

- User selects a PDF
- App reads bytes from the content URL
- App performs an HTTP upload to Supabase Storage
- App saves the resulting CV URL in Firestore

This ensures that CV state is centralized in the user profile for reuse across applications.

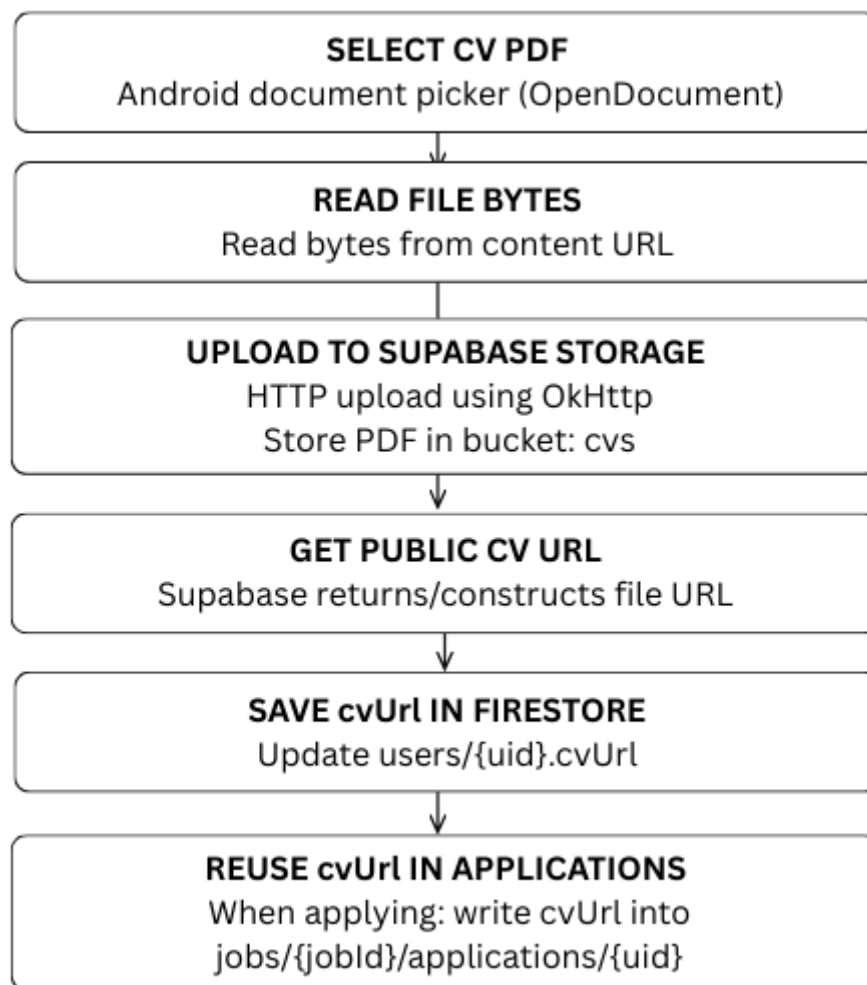


Figure 11. CV upload and reference handling workflow

3.6.2 Using CV Reference in Applications

Before applying to a job, the app checks the user profile for a valid cvUrl. If present, the same URL is copied into the application document. This avoids dependence on a second live lookup at review time and keeps each application record self-contained.

3.7 Access Control and Data Security

Since the system handles identity data, professional data, academic data and applicant documents, it requires clear access control. Firebase Authentication provides signed-in identity, but access enforcement depends on Firestore Security Rules and storage policy configuration.

3.7.1 Summary of Security Architecture

Security model relies on:

1. Authentication through Firebase Auth (email/password and Google sign-in)
2. Authorization through role and ownership fields stored in Firestore
3. Policy enforcement through Firestore Security Rules and secure storage configuration

In a secure deployment, rules should ensure:

- Users can only read/write their own user document
- Companies can only create and manage jobs they own
- Applicants can only create applications for themselves
- Companies can only read applications for jobs they own

Table 10. Security Architecture

Resource	Who can read	Who can write	Rule condition (summary)
users/{uid}	Owner (uid)	Owner (uid)	Request.auth.uid == uid
jobs/{jobId}	All signed-in users	Company owner	Write: request.auth.uid == companyId
jobs/{jobId}/applications/{uid}	Applicant and job owner	Applicant only	Read: applicantUid == auth.uid OR job.companyId == auth.uid; write: applicantUid == auth.uid
CV storage objects	Owner (private) or controlled access	Owner (upload only)	Private bucket + authenticated reads or signed URLs

3.7.2 CV Storage Sensitivity

CVs are personally sensitive artifacts. A robust setup should:

- Store CVs in private buckets
- Provide controlled access (signed URLs or authenticated reads)
- Avoid embedding privileged storage keys inside the Android application

3.8 Implementation of Frontend and User Interface Logic

The frontend is implemented using Jetpack Compose and follows a screen-based structure aligned with the primary flows: authentication, role-driven navigation, job operations, application submission and profile management.

3.8.1 Routing and Navigation

Routes are defined in MainActivity.kt using Compose Navigation. The main navigation destinations include:

- Login
- Signup
- Home
- Profile
- Jobs
- Job_create
- Applicants/{jobId}
- Recommendations
- Action_plan
- Skill_impact

Navigation is coordinated with authentication state. After successful authentication, the user is routed to home, where role-specific navigation paths become available.

Table 11. Navigation routes and role access

Route	Screen	Role access	Purpose
Login	Login screen	Applicant + Company	Authenticate existing users
Signup	Signup screen	Applicant + Company	Register new users and set initial role/profile
Home	Home dashboard	Applicant + Company	Role based actions and navigation entry
Profile	Profile screen	Applicant + Company	View and edit user profile fields
Jobs	Jobs listing	Applicant + Company	Applicant: browse jobs; company: view own jobs
Job_create	Job create screen	Company only	Create and publish a job post
Applicants/{jobId}	Applicants list	Company only	View applications for a selected job
Action_plan	Action plan screen	Applicant only	Generate and view the learning/action plan
Skill_impact	Skill impact screen	Applicant only	View skill impact/skill gap insights and progress focus

3.8.2 UI Design Approach

The UI uses standard Compose components (Scaffold, Card, LazyColumn, TextField, Button). The implementation focuses on predictable interactions: form input for auth and job creation, list-based browsing for jobs and applicants, and explicit buttons for actions such as apply, upload CV, and view applicants.

3.8.3 Profile Editing Strategy

Profile editing is implemented as field-level updates: one field can enter edit mode at a time, and confirmed edits are written directly to Firestore. This results in small, atomic updates and makes it easier to reason about profile state changes.

CV parsing is treated as a structured profile update step: when the user uploads a CV PDF, the app parses the document into text and derives normalized profile signals then writes those extracted fields back to `users/{uid}` as Firestore fields keeping manual edits consistent.

Table 12. Profile editing rules and firestore updates

Rule	UI behaviour	Firestore effect
One field at a time	Only one profile field enters edit mode	Prevents conflicting edits
Confirm save	User confirms the edited value	Update only the edited field in <code>users/{uid}</code>
Cancel edit	User exits edit mode without saving	No write is performed
Atomic updates	Small, field-level changes	Partial updates (merge) instead of overwriting full document
Name lock	If <code>nameLocked</code> is true, name edit is disabled	Keeps identity stable in UI
CV link update	After upload, <code>cvUrl</code> is refreshed on profile	Update <code>users/{uid}.cvUrl</code> after successful upload

3.9 Backend Integration and Cloud Service Logic

The backend functionality was achieved through SDK integration rather than a custom API. All persistent operations are performed using Firebase SDK calls (Auth + Firestore) and CV storage uses an HTTP upload to Supabase Storage.

3.9.1 Firebase Authentication and Firestore Operations

The system performed the following core operations:

- User registration and login (Firebase Auth)
- Role and profile persistence (Firestore users collection)
- Job creation and job retrieval (Firestore jobs collection)
- Application creation and applicant retrieval (applications subcollections)

Realtime updates are supported through Firestore listeners where applied, allowing the UI to stay consistent with the current database state.

3.9.2 Supabase Storage Operations

CV upload is performed via OkHttp to Supabase Storage. The stored object was referenced through a URL that was written into Firestore. CV viewing was delegated to external PDF handlers using URL opening intents.

3.10 AI Decision Support Implementation

The AI component of the system was implemented as an explainable decision support layer that connects CV intake, skill extraction, role-skill matching, readiness scoring and prioritized skill-gap recommendations inside the same Android and Firebase cloud workflow. Instead of relying on a separate inference server, the project uses an offline training and offline export pipeline to generate lightweight artefacts that are bundled into the Android application and consumed at runtime through Kotlin modules.

3.10.1 AI pipeline and Exported Artefacts

The AI pipeline was implemented as an offline training and export workflow that produced reusable artefacts for the mobile application. The pipeline used roles.json as the role/skill knowledge base, trained the model and stored outputs under out/, then exported a skill importance mapping that was shipped with the Android project. The main artefacts produced and used by the app are:

- roles.json (role definitions and skill sets)
- out/skill_priority_map.json (exported skill priority weights derived from the trained model)
- out/recommender.joblib and out/meta.json (training outputs retained for reproducibility in the pipeline)

3.10.2 Role Catalogue, Matching and Readiness Scoring

In Career Guidance App, the role catalogue was loaded from app assets through RolesRepository.kt and RoleCatalogDataSource.kt. The required and optional skill sets used for matching were provided through these definitions. Ranked role recommendations were computed by RecommendationEngine.kt by comparing the applicant's normalized skills against each role's required and optional skills. For the goal role selected by applicant the detailed matching state was generated by GoalPlanEngine.kt, including matched skills, missing skills and a readinessScore. The ranking of missing skills was guided by SkillPriorityRepository.kt with weights read from the exported skill_priority_map.json artefact.

3.10.3 CV-to-Skill Extraction and Normalization

CV processing was implemented as a profile driven workflow in which an uploaded PDF was converted into structured signals usable by the matching engines. Text was extracted from PDF bytes in CvParser.kt, and structured profile fields like skillsRaw and skillsNormalized were derived using the role/skill vocabulary and SkillNormalizer so that matching consistency was maintained. The extracted profile signals were written to Firestore under users/{uid} and a verifiable profile state was maintained for the decision support calculations.

3.10.4 User Facing AI Outputs and Persistence in Firestore

AI outputs were displayed through the recommendation and planning UI screens; RecommendationsScreen.kt, ActionPlanScreen.kt, SkillImpactScreen.kt through their corresponding ViewModels where user profile data was loaded from Firestore and decision support results were computed using the matching engines. Persisted state was kept minimal; users/{uid} was used to store the current extracted signals and users/{uid}/plans/goalPlanProgress was used to store the ongoing learning-plan completion state (completedSkills). Explainable guidance was supported because the readiness score and prioritized gaps could

be reproduced from the stored profile signals together with roles.json and skill_priority_map.json.

4 RESULTS

4.1 Working Functions

This project was implemented as an Android Jetpack Compose prototype with Firebase Authentication and Firebase database as the primary backend services. Supabase Storage was used for storing applicant CV files. The main flows were validated through an end-to-end walkthrough of the implemented code paths, covering sign-up, login, role based navigation and UI routing. Firestore CRUD operations support user profile persistence, job creation and listing, and application submission under `jobs/{jobId}/applications` with duplicate prevention and applied-status tracking. The AI-driven decision support layer operates inside the same app flow by using packaged artefacts.

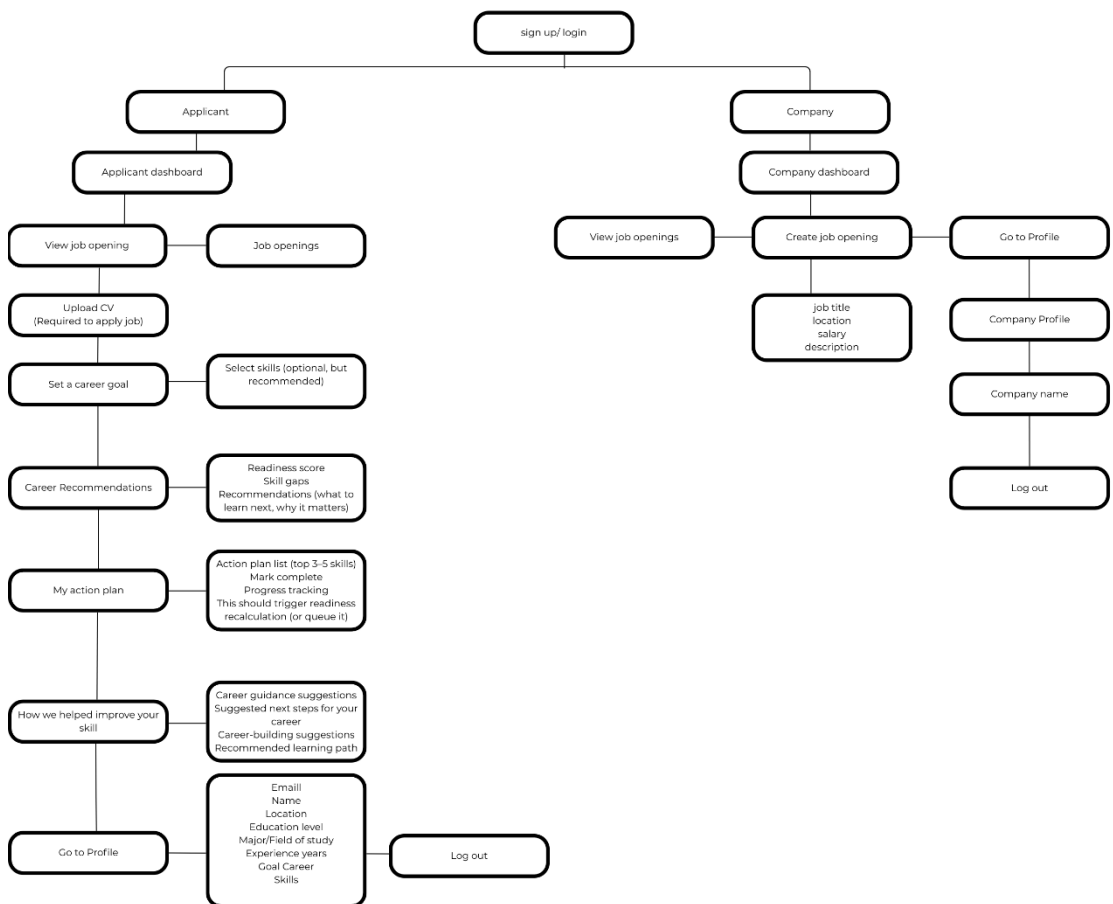
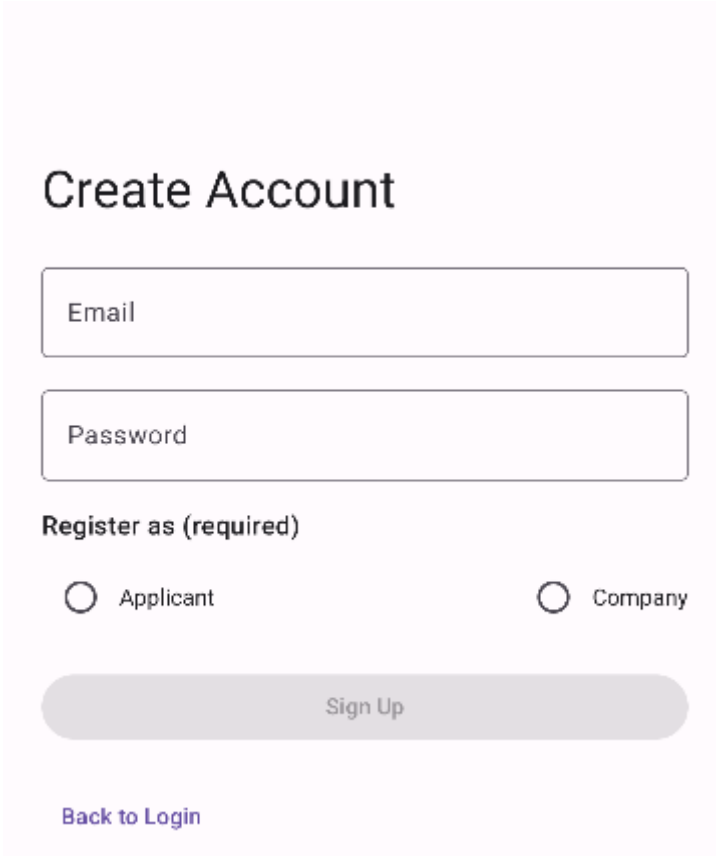


Figure 12. Main navigation and integrated flow in MainActivity(Navigation routes)

The application supports two roles: applicant and company. The main user flows covered by the implementation are:

- Account creation (email/password) with explicit role selection
- Login (email/password) and login via Google Sign-In (with restrictions)
- Role based home dashboard
- Profile viewing and profile field editing
- Company job posting
- Job browsing for applicants
- Job application submission (with CV required)
- Company side applicant viewing for each job
- Opening CV and external profile links via Android intents
- AI-driven recommendations and skill matching for applicants



The image shows a 'Create Account' screen with a light pink background. At the top, the title 'Create Account' is displayed in a large, bold, black font. Below the title are two input fields: 'Email' and 'Password', each with a light purple border and a small purple icon on the left. Under the 'Password' field, the text 'Register as (required)' is shown. Below this text are two radio button options: 'Applicant' and 'Company', each with a small purple circle icon. At the bottom of the form is a large, rounded, light purple button with the text 'Sign Up' in a darker purple font. Below the button is a link 'Back to Login' in a purple font.

Figure 13. Signup screen with role selection (UserRole.kt, SignupScreen.kt)

Navigation is handled from MainActivity using a Compose NavHost with routes for login, signup, home, profile, jobs listing, job creation, applicants list screen tied

to a selected jobId, recommendations, action_plan, set career goals and skill_impact. Firestore snapshot listeners are used in the job listing and applicant listing to reflect updates dynamically when new jobs are posted or new applications are added. The UI includes basic loading states and user feedback through Toast messages for error states such as missing required fields, missing authentication, or missing CV. AI-driven decision support is integrated by processing the uploaded CV into normalized skill signals using roles.json and skill_priority_map.json to generate explainable role recommendations, readiness scores and prioritized skill-gap guidance.

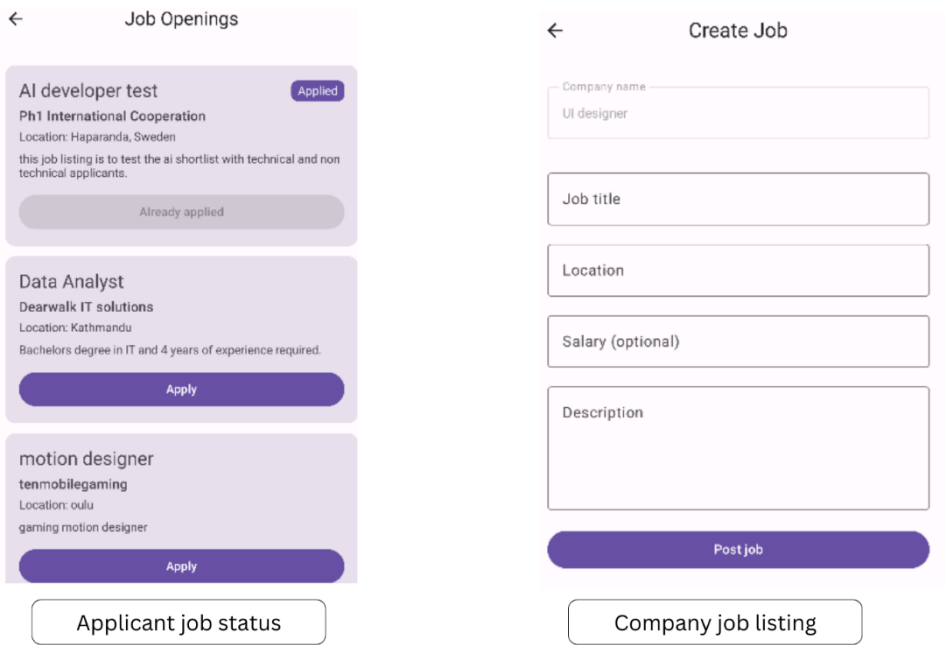


Figure 14. Job application status in applicant and company job listing views

4.2 AI Decision Support Behaviour and Action Plan Engine

This career guidance prototype follows the AI-driven decision support workflow that turns the applicant’s CV and profile state into changing career recommendations, readiness scoring and prioritized skill-gap guidance. In the implemented flow, roles are loaded from roles.json and skill priorities from skill_priority_map.json and the applicant’s normalized skills are compared against role requirements to compute matched skills, missing skills and a readiness score for the selected goal role. Overall, the system behaved

consistently by producing deterministic, inspectable results based on the stored profile signals and artefact-based weighting rather than opaque adaptation.

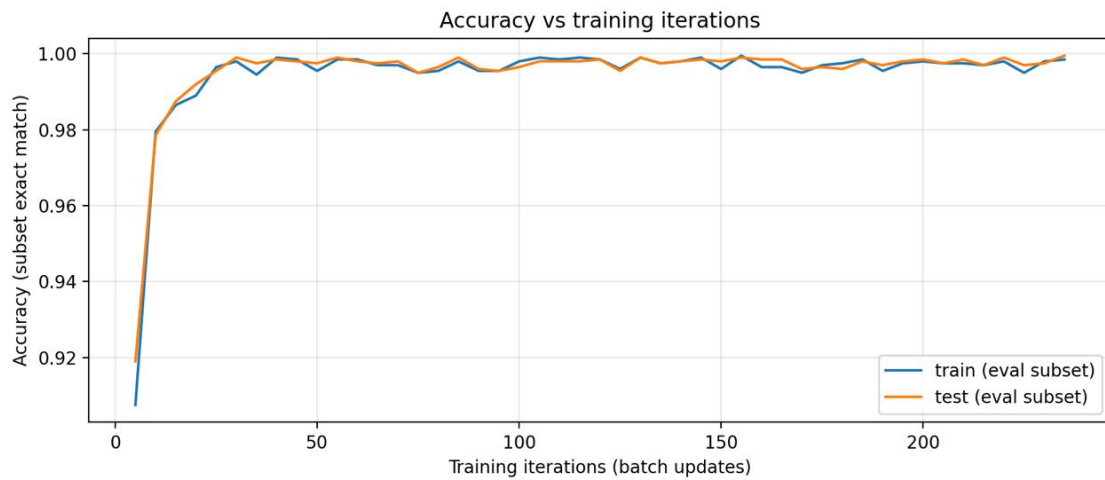


Figure 15. Accuracy vs training iterations

4.3 Role Selection

Role selection is represented by UserRole (applicant/company) during signup and stored as a string field role in Firestore under users/{uid}. After login, HomeScreen fetches this role and switches the dashboard content accordingly:

- Company accounts see a company dashboard with an option to create job openings, view list of applicants who applied for jobs
- Applicant accounts see an applicant dashboard with an option to upload a CV, view job openings, set a career goal, get career recommendations, view action plan, improve skill gap recommendations

Table 13. Role based actions enabled for applicant and company users

Action	Applicant	Company
View jobs	Yes	No
Create job post	No	Yes
View own job posts	No	Yes
Apply to job	Yes	No
View applicants for own job	No	Yes

Upload CV	Yes	No
View CV link(cvUrl) for applicants	No	Yes(read-only)
Edit own profile	Yes	Yes
View other users' profile	No	No
Logout	Yes	yes

The job listing behaviour also depends on role:

- Company accounts only see jobs where companyId equals their uid
- Applicant accounts see all jobs ordered by createdAt descending.

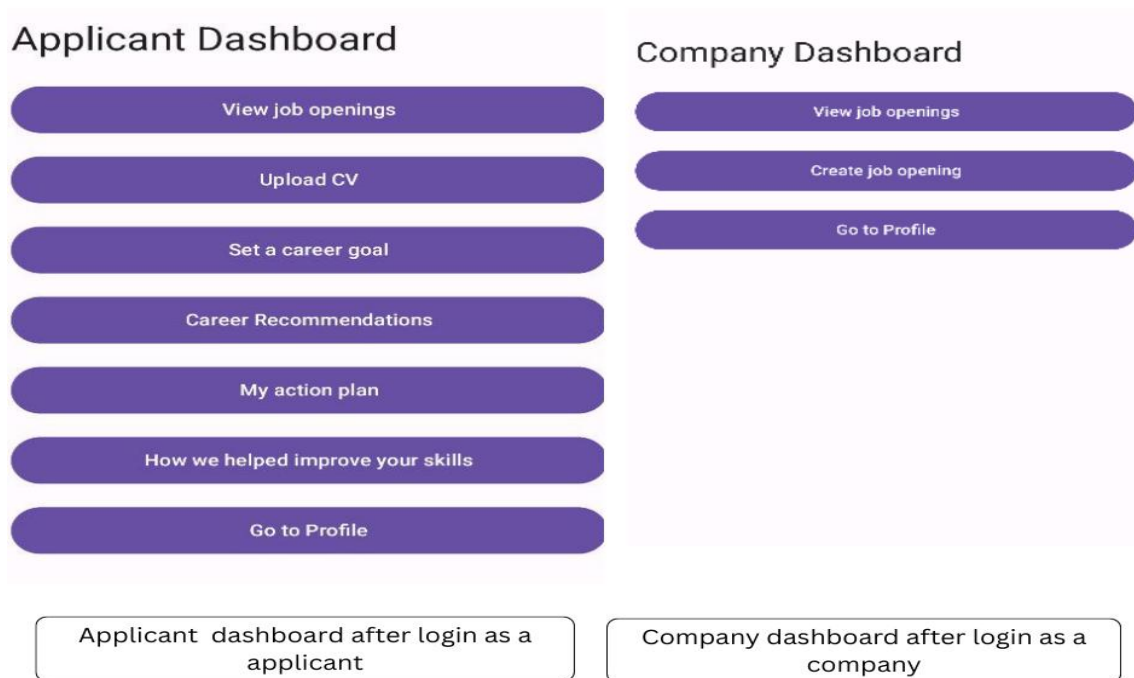


Figure 16. Role based home dashboards for applicant and company users

Google Sign-In is supported in MainActivity.kt, but the code explicitly prevents company accounts from using Google Sign-In by signing them out if role equals "company". This is a functional constraint implemented at runtime and depends on the role field being present in Firestore.

```

private val googleLauncher =
    registerForActivityResult(ActivityResultContracts.StartIntentSenderForResult()) { result ->
        val credential = Identity.getSignInClient(this)
            .getSignInCredentialFromIntent(result.data)

        val token = credential.googleIdToken
        val firebaseCred = GoogleAuthProvider.getCredential(token, null)

        auth.signInWithCredential(firebaseCred)
            .addOnSuccessListener {
                val user = auth.currentUser ?: return@addOnSuccessListener
                val uid = user.uid

                // Block company accounts from Google sign-in
                firestore.collection("users")
                    .document(uid)
                    .get()
                    .addOnSuccessListener { doc ->
                        val role = doc.getString("role")
                        if (role == "company") {
                            auth.signOut()
                            Toast.makeText(
                                this,
                                "Company accounts cannot use Google sign-in. Please log in with email and password.",
                                Toast.LENGTH_LONG
                            ).show()
                        }
                    }
            }
    }

```

Figure 17. Google sign-In is blocked for company accounts (MainActivity.kt)

4.4 Real World Observation from Alpha Testing

This project does not include a controlled user study, and no formal usability test dataset is stored inside the repository. It describes what can be observed during a realistic role based walkthrough of the implemented features which means what happens when an applicant uses the system versus a company uses the system. For Alpha testing, following devices and emulators were used:

- Pixel 9 Pro XL API 36.0 emulator in MacBook Air 13-inch (2025) through Android Studio Otter 3
- Pixel 8 Pro API 36.1 emulator in Acer Aspire A315-24P through Android Studio Otter 3
- Pixel 9 Pro XL API 35 emulator in ASUS VIVOBooki7
- Samsung SM-A155F API 36.0
- Redmi Note 9

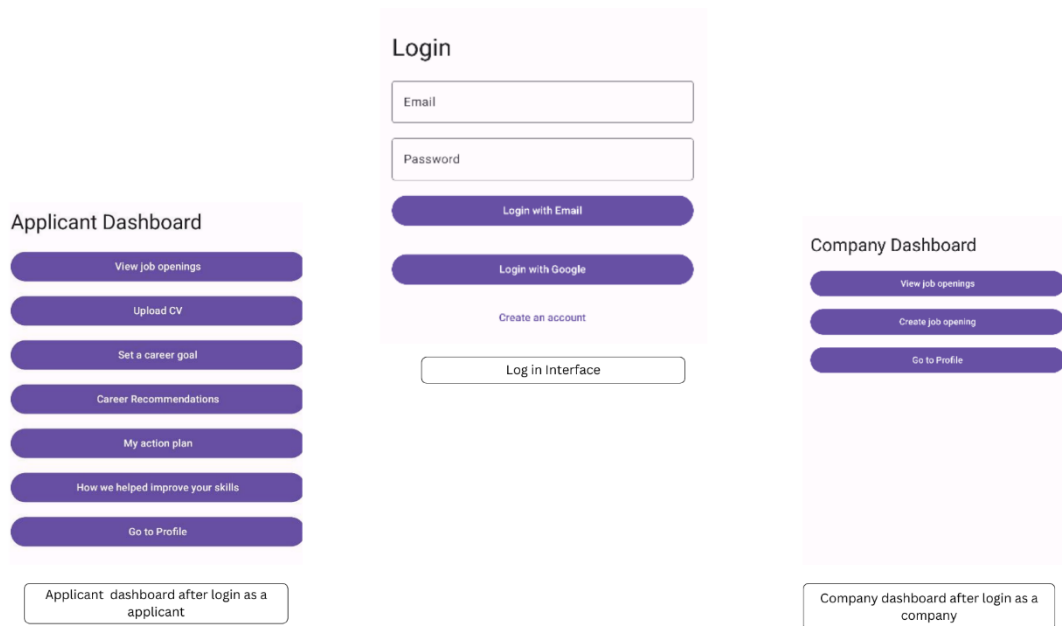


Figure 18. Login screen and successful navigation to home after authentication

Observed during a typical walkthrough:

- A user can sign up with email and password only after choosing an account type (applicant or company) and on success the app creates users/{uid} with email, role, and createdAt then routes the user to the home screen.
- Google sign-in is available on the login screen, but if the signed-in account is detected as role = company in users/{uid}.role, the app signs out and shows a message that company accounts cannot use Google sign-in.
- Role based navigation is enforced from the home screen: companies can create job openings and view applicants, while applicants can upload/update a CV, set a career goal, and open the recommendations, action plan, and skill impact screens.
- An applicant can set a career goal from the home screen using a goal selection dialog, and the selected value is saved to users/{uid}.goalRole with a confirmation message.
- An applicant can browse job openings, but applying requires a CV URL to exist in their user profile document; if no CV is present, the UI blocks the application action and shows a message instructing the user to upload a CV first.

- Once an applicant applies to a job, the job card displays an “Applied” status label and the apply button becomes disabled for that job (based on Firestore checks).
- A company user can create a job, and the job creation form attempts to auto-fill the company name from users/{uid}.name; if the company name is empty, the app blocks posting and asks the user to set it in the profile first.
- A company user can open an applicants list for a specific job and access the stored CV links via an “Open CV” action.
- An applicant can get the AI-driven recommendations, action plan and skill impact features and plan progress is saved under users/{uid}/plans so results remain visible across sessions.

4.5 User Experience and Outcomes

In about 50 to 60 testing sessions among developers, Career Guidance App projected following outcomes:

- Stable operation with no major failures in the core flows like sign-up, login, role based navigation, job posting, job browsing, applying and applicant review
- Correct implementation of role based behaviour and controlled access
- Consistent end-to-end persistence through Firebase Auth, Firestore and Supabase Storage
- Meaningful guidance assistance given by AI decision support components
- A straightforward and useful interface that testers thought was intuitive career guidance app

Besides the presence of small trial volume, the informal evaluation proved the feasibility of prototype by successfully demonstrating the combination of role based job marketplace workflows, CV-based decision support and explainable AI guidance within a single Android application. The qualitative findings confirmed that the app acted as designed.

5 DISCUSSION

The purpose of this thesis was to develop a realistic Android mobile application based on the architecture design that can facilitate career guidance and recruitment processes based on a role based model (applicant and company). After registration, the selected role is stored in the cloud database and is used to control navigation and feature access inside the app.

The implemented system demonstrates a practical recruitment loop. Companies create job openings, applicants browse available jobs, and applications are stored so that companies can review candidates. Cloud backend services are used for authentication and data storage, and a CV upload workflow is supported through cloud storage where the uploaded document link is saved to the database and reused during job applications.

5.1 Discussion in Relation to Research Questions

RQ1: How can a role based career guidance system be effectively implemented as an android mobile application using cloud backend service and an integrated AI decision-support pipeline?

The research question is whether a multi-role Android application (applicant and company) can be implemented in a practical way with the use of cloud services as an essential part of core operations such as authentication, data storage, and document handling. Role based behaviour in this project is achieved by saving the chosen role during registration (applicant or company) and using the role to control which features and screens can be accessed after logging in. For example, applicants are guided to complete their profile, upload a CV, browse job advertisements, and apply for jobs, whereas companies are guided to create job openings and review applicants for each job.

This implementation relies on the cloud backend. Authentication provides a secure identity (user uid), and the database stores user role and profile data, job

posts, and applications. One of the most important features supporting the recruitment workflow is CV intake: applicants upload a PDF file, and the system saves a CV link so that companies can review the CV when evaluating applicants. This design shows that the selected Android and cloud technologies can support a complete recruitment loop as part of a mobile workflow, which is important for an effective career guidance system.

In the case of the integrated AI decision-support pipeline, explainable matching, readiness scoring, and skill-gap recommendations are included as part of the thesis objectives. In addition to the role based workflow and structured data collection, the thesis also includes an AI decision-support pipeline as a project artifact (`career_ai_training`), which prepares recommendation outputs based on role requirements. The pipeline uses role definitions (required and optional skills with weights and experience-based focus) to train a recommendation model and export Android-consumable outputs (such as a skill priority mapping) that can be used inside the mobile application. This supports skill guidance by translating role requirements into understandable learning priorities for applicants. However, the current AI component is mainly demonstrated through structured role–skill definitions and controlled/synthetic training samples, and therefore large-scale validation on real-world CV datasets and different CV formats remains a future improvement for strengthening job–CV matching reliability.

RQ2: How do cloud-based authentication and database management contribute to security, scalability, usability and controlled access to AI-generated insights in a career guidance application?

The research question explores the use of cloud services to enhance the application in terms of secure access, reliable data storage, improved user experience, and controlled access to sensitive information (including AI-related outputs). Cloud-based authentication reduces the risk of insecure local account management. Every user operation is linked to a unique uid, which enables data to be stored and retrieved in a structured form and supports role based control.

Cloud database management also offers scalability and maintainability by ensuring that user profiles, job postings, and applications exist on a centralized

platform rather than on-device storage. In terms of usability, this means users can log in on different devices without losing data, new job posts can be displayed immediately in job lists, and companies can view applicant lists without manual data transfer. The database also allows rule-based organization of data, for example storing applications under each job, which makes retrieval and review practical for company users.

The importance of controlled access is particularly due to the fact that the system deals with sensitive personal information as well as CV documents. The design relies on role based access: companies should only manage their own job posts and view applications submitted to those job posts. These restrictions should be enforced even if the UI is bypassed in a production-level system by implementing cloud security rules. Similarly, if AI-generated insights are added (readiness scores, matching explanations), they should be stored and displayed only to roles that have permission to access them. In addition, since the thesis includes an AI decision-support pipeline that produces guidance outputs (for example, skill priority recommendations), the same cloud-based access model can be used to store and protect those AI results. This implies that applicant-facing AI insights should be accessible only to the applicant account linked to the uid, while company access should remain limited to permitted recruitment-related information. This combination of authentication, role storage, and structured database collections provides a strong foundation, but strict rule validation and privacy-conscious document access are still necessary for real deployment.

RQ3: How does the MVVM architectural pattern support maintainability, clarity and reliable integration of AI-driven features in a role based android application?

The research question examines how the MVVM architectural pattern can support the development of a role based Android application by improving maintainability, clarity, and the reliability of integrating AI-driven features. MVVM supports separation of concerns by keeping UI rendering in composable screens, while business logic and state management are handled through ViewModels. This separation can reduce the complexity of the UI layer and make the application easier to understand, extend, and maintain over time.

In the implemented system, MVVM principles are applied in parts of the project, especially in authentication-related flows and in handling UI state. This provides clearer structure because UI screens can focus on presenting data and responding to user input, while ViewModels can manage loading states, validation, and communication with cloud services. However, the prototype also includes areas where database operations and logic are placed directly inside composables. While this approach can speed up development during prototyping, it may reduce maintainability because it increases coupling between UI and data operations and makes testing more difficult.

In relation to AI-driven integration, architectural separation becomes even more important. AI workflows may require multiple steps such as preparing input features, invoking recommendation logic, handling outputs, and presenting explanations to the user. When these steps are placed behind ViewModels and repository/service classes, the system becomes more reliable and easier to modify without breaking the UI. Since the thesis includes an AI decision-support pipeline that exports Android-consumable outputs (such as skill priority guidance), these outputs can be integrated in a maintainable way by loading them through a data layer and exposing them to the UI as state. Therefore, MVVM supports a clearer structure for integrating AI features, but more consistent use of MVVM across all screens and better layering (UI, domain, data) would further improve maintainability and long-term scalability.

5.2 Critical Evaluation and Limitations

The results are mainly validated through scenario-based testing performed by the developer rather than a formal user study. This supports functional verification of requirements, but it limits how strongly usability conclusions can be generalised to a wider user population.

Performance was not measured with detailed latency metrics or load testing. Therefore, responsiveness is discussed mainly in functional terms, and behaviour under heavy concurrency, poor network conditions, or a wide range of devices remains outside the tested scope.

A further limitation concerns the AI decision-support pipeline. The current AI artifact is based on structured role–skill definitions and controlled or synthetic training samples, rather than large-scale real-world CV datasets with verified labels. Therefore, the pipeline demonstrates feasibility and explainability of skill guidance, but it does not provide statistically grounded accuracy results across diverse CV formats, terminology differences, and real hiring requirements. Broader validation using more varied CV inputs and role requirement data would be needed for stronger conclusions about recommendation quality.

There are also report-alignment issues that should be corrected for academic consistency. For example, any template-based sections that describe features not implemented (such as an adaptive learning engine) should be rewritten so that the document accurately describes the role based recruitment workflows that were developed.

Finally, CV handling is the most sensitive area from a security and privacy perspective. In a prototype, direct client-side uploads demonstrate the workflow, but a production system should enforce strict access control, use private storage, and avoid embedding privileged storage keys in the mobile application. In addition, client-side storage credentials must not be embedded in production apps, because they can be extracted from the APK and misused.

5.3 Ethical and Data Protection Considerations

The application makes use of the personal profile information as well as the CV documents making the application. responsible design as data security and privacy. The prototype follows a data minimisation strategy in which the fields needed in the recruitment workflow are obtained and it alone is collected. authentication recognizes an action as having a validated user identity. However, CV files are very sensitive and should be preserved through confidential depository, restricted access. mechanisms such as signed URLs, and clear retention policy in real deployments.

Since the thesis includes AI-based guidance outputs, ethical responsibility also includes transparency and careful interpretation of recommendations. AI outputs

such as skill priorities or readiness-related feedback should be presented as supportive guidance, not as final decisions. In real deployment, users should understand what the AI output means, what data it is based on, and how it can be corrected or updated.

Sample data aids in reducing the ethical risk when developing or testing. In a real-world setting, the system would need clear user consent practices, the explanation of which is clear. whom do the applicant records allow to be seen and guard against any accidental disclosure or disclosing of individual files over a prolonged duration of time.

5.4 Practical Implications

The prototype demonstrates that role separation can enhance usability as well as security in mobile recruitment tools. Applicants observe applicant relevant actions like uploading a CV, browsing jobs and applying, while companies see company-relevant actions such as creating jobs and reviewing candidates. This helps in minimizing confusion and the possibility of misuse through restricting the role of each position in the interface.

Managed cloud services are a feasible option with regard to small development teams. Without the construction, authentication can be deployed as well as database management. and with a full-custom backend, which gives more time to concentrate on the core user. processes and continuous development through feedback.

In addition, the AI decision-support pipeline adds practical value by turning role requirements into understandable learning priorities. This supports applicants in planning skill development and creates a foundation for future expansion into more detailed explainable matching outputs.

5.5 Conclusion

The study effectively fulfills its fundamental objective, as the project indicates that a role based career guidance and recruitment approach can be implemented successfully. By combining an Android application with cloud backend services, the workflow can be realised through an organised data model and secure identity-based access. In addition, the AI training and export artifact demonstrates how role–skill definitions can be transformed into explainable skill guidance outputs that are suitable for mobile use.

The prototype provides a viable foundation for role separation, job marketplace interactions, and document-based applications. The major gaps are associated with readiness for production, including greater security of document storage, increased MVVM stratification, broader evaluation, and broader validation of AI guidance quality. Addressing these areas would make the system more stable, serviceable and closer to the requirements of real-life deployment, while still being aligned with the thesis objective and scope.

5.6 Future Work

There are several improvements that would be enhance the system and bring it further towards production.

Secure document storage: Move CV upload and access to a design that does not expose privileged keys in the client. A suitable approach is to use a secure backend or serverless function to generate time-limited signed URLs and enforce role based access policies.

Consistent MVVM layering: Introduce repositories and a dedicated data layer to centralise Firestore and storage logic. This reduces duplicated code, improves testability and supports long-term maintainability.

Stronger backend rule validation: Add negative tests that attempt unauthorised reads or writes (for example, reading another user’s profile or modifying jobs not owned by the company) and confirm that backend rules block them.

Broader evaluation: Conduct small user studies with applicants and company users and test across multiple devices and network conditions to identify usability gaps and reliability issues.

AI evaluation and improvement: Validate the recommendation outputs using more diverse CV examples, terminology variations, and role requirement datasets, and report the results clearly.

Explainable guidance in the app: Extend the AI outputs from general skill priorities into clearer role-specific explanations (matched skills, missing skills, and why each recommendation is suggested).

Secure storage for AI results: If AI outputs are saved in the backend, store them under user-linked collections and protect them with strict role based rules, similar to profile and CV access control.

Robust CV processing: Improve handling of different PDF CV layouts and ensure that any extracted insights remain privacy-preserving and securely accessible.

Report alignment: Ensure the methodology and implementation descriptions match the final storage and architecture decisions and remove or rewrite any remaining template sections that do not represent the implemented system.

References

- [1] P. Beer and R. H. Mulder, "The effects of technological developments on work and their implications for continuous vocational education and training: A systematic review," *Front. Psychol.*, vol. 11, 2020, doi: 10.3389/fpsyg.2020.00918.
- [2] OECD, "Empowering the Workforce in the Context of a Skills-First Approach," *OECD Skills Studies*, 2025.
- [3] M. Bone, E. González Ehlinger, and F. Stephany, "Skills or degree? The rise of skill-based hiring for AI and green jobs," *Technol. Forecast. Soc. Change*, vol. 214, 2025, doi: 10.1016/j.techfore.2025.124042.
- [4] B. Đorđević, S. Milanović Zbiljić, and M. Radosavljević, "Impact of the Digital Skills on Employability: Cross-Sectional Analysis," *Economies*, vol. 13, no. 7, 2025, doi: 10.3390/economies13070196.
- [5] N. Türker and E. Üngüren, "Digital footprints and recruitment: an experimental study on the impact of social media content on hiring decisions," *Front. Psychol.*, vol. 16, 2025, doi: 10.3389/fpsyg.2025.1693850.
- [6] G. D. Marin and C. Nilă, "Branding in social media. Using LinkedIn in personal brand communication: A study on communications/marketing and recruitment/human resources specialists perception," *Social Sciences and Humanities Open*, vol. 4, no. 1, 2021, doi: 10.1016/j.ssaho.2021.100174.
- [7] J. K. H. Chiang and H. Y. Suen, "Self-presentation and hiring recommendations in online communities: Lessons from LinkedIn," *Comput. Human Behav.*, vol. 48, 2015, doi: 10.1016/j.chb.2015.02.017.

- [8] C. Amanova, S. B. Ozgur, F. Msilu, and F. Demir, "Demystifying the User Experience: A Case Study on Online Job Search Engines," *American Journal of Qualitative Research*, vol. 5, no. 2, 2021.
- [9] A. Deshmukh and A. Raut, "Enhanced Resume Screening for Smart Hiring Using Sentence-Bidirectional Encoder Representations from Transformers (S-BERT)," *International Journal of Advanced Computer Science and Applications*, vol. 15, no. 8, 2024, doi: 10.14569/IJACSA.2024.0150825.
- [10] C. Rigotti and E. Fosch-Villaronga, "Fairness, AI & recruitment," *Computer Law and Security Review*, vol. 53, 2024, doi: 10.1016/j.clsr.2024.105966.
- [11] M. White, "Information anywhere, any when: The role of the smartphone," *Business Information Review*, vol. 27, no. 4, 2010, doi: 10.1177/0266382110390651.
- [12] P. Mell and T. Grance, "The NIST definition of cloud computing," in *Cloud Computing and Government: Background, Benefits, Risks*, 2011. doi: 10.1016/b978-0-12-804018-8.15003-x.
- [13] M. E. Frame, J. Kaiser, J. Kegley, J. Armstrong, and B. Schlessman, "Impacts of decision support systems on cognition and performance for intelligence-gathering path planning," *Military Psychology*, vol. 36, no. 3, 2024, doi: 10.1080/08995605.2023.2178210.
- [14] D. Shin, "The effects of explainability and causability on perception, trust, and acceptance: Implications for explainable AI," *International Journal of Human Computer Studies*, vol. 146, 2021, doi: 10.1016/j.ijhcs.2020.102551.
- [15] H. Al-Dossari, Z. Al-Qahtani, F. A. Nughaymish, M. Alkahlifah, and A. Alqahtani, "CareerRec: A Machine Learning Approach to Career Path Choice for Information Technology Graduates," *Engineering, Technology and Applied Science Research*, vol. 10, no. 6, 2020, doi: 10.48084/etasr.3821.

- [16] N. Dawson, M. A. Williams, and M. A. Rizoio, "Skill-driven recommendations for job transition pathways," *PLoS One*, vol. 16, no. 8 August, 2021, doi: 10.1371/journal.pone.0254722.
- [17] F. Tang, R. Zhu, F. Yao, J. Wang, L. Luo, and B. Li, "Explainable person–job recommendations: challenges, approaches, and comparative analysis," 2025. doi: 10.3389/frai.2025.1660548.
- [18] Y. Sun, F. Zhuang, H. Zhu, Q. He, and H. Xiong, "Cost-effective and interpretable job skill recommendation with deep reinforcement learning," in *The Web Conference 2021 - Proceedings of the World Wide Web Conference, WWW 2021*, 2021. doi: 10.1145/3442381.3449985.
- [19] R. Parasuraman and V. Riley, "Humans and automation: Use, misuse, disuse, abuse," *Hum. Factors*, vol. 39, no. 2, 1997, doi: 10.1518/001872097778543886.
- [20] Firebase, "Firebase Control Access," Firebase Control Access Documentation. Accessed: Jan. 15, 2026. [Online]. Available: <https://firebase.google.com/docs/auth/admin/custom-claims>
- [21] Firebase, "Firebase Role Based Access," Firebase Role Based Access Documentation. Accessed: Jan. 15, 2026. [Online]. Available: <https://firebase.google.com/docs/firestore/solutions/role-based-access>
- [22] Firebase, "Firestore Security Rules ," Firestore Security Rules Documentation. Accessed: Jan. 02, 2026. [Online]. Available: <https://firebase.google.com/docs/firestore/security/get-started>
- [23] Android Inc, "Android ViewModel Overview," Android ViewModel Overview Documentation. Accessed: Jan. 15, 2026. [Online]. Available: <https://developer.android.com/topic/libraries/architecture/viewmodel>

- [24] S. Kim and H. J. Mun, "Design and development of a self-diagnostic mobile application for learning progress in non-face-to-face practice learning," *Applied Sciences (Switzerland)*, vol. 11, no. 22, 2021, doi: 10.3390/app112210816.
- [25] Firebase, "Firebase Authentication," Firebase Authentication Documentation. Accessed: Dec. 26, 2025. [Online]. Available: <https://firebase.google.com/docs/auth>
- [26] K. Milojković, M. Živković, and N. Bačanin Džakula, "Agile Multi-user Android Application Development With Firebase: Authentication, Authorization, and Profile Management," 2024. doi: 10.15308/sinteza-2024-405-412.
- [27] S. Talegaon and R. Krishnan, "Administrative models for role based access control in android," *Journal of Internet Services and Information Security*, vol. 10, no. 3, 2020, doi: 10.22667/JISIS.2020.08.31.031.
- [28] P. R. Saraf, "A Review on Firebase (Backend as A Service) for Mobile Application Development," *Int. J. Res. Appl. Sci. Eng. Technol.*, vol. 10, no. 1, 2022, doi: 10.22214/ijraset.2022.39958.
- [29] P. Weichbroth, "Usability Testing of Mobile Applications: A Methodological Framework," *Applied Sciences (Switzerland)*, vol. 14, no. 5, 2024, doi: 10.3390/app14051792.
- [30] J. Ryser and M. Glinz, "A Scenario-Based Approach to Validating and Testing Software Systems Using Statecharts," in *12th International Conference on Software and Systems Engineering and their Applications (ICSSEA'99)*, 1999.
- [31] P. Skoković and M. Rakić-Skoković, "Requirements-based testing process in practice," *International Journal of Industrial Engineering and Management*, vol. 1, no. 4, 2010, doi: 10.24867/ijiem-2010-4-018.

- [32] P. Runeson, M. Höst, A. Rainer, and B. Regnell, *Case Study Research in Software Engineering*. 2012. doi: 10.1002/9781118181034.
- [33] ENISA, "Privacy and data protection in mobile applications," *Enisa*, no. November, 2017.
- [34] D. Falessi *et al.*, "Empirical software engineering experts on the use of students and professionals in experiments," *Empir. Softw. Eng.*, vol. 23, no. 1, 2018, doi: 10.1007/s10664-017-9523-3.
- [35] Android Inc., "Compose AI Architecture," Compose UI Architecture Documentation. Accessed: Dec. 28, 2025. [Online]. Available: <https://developer.android.com/develop/ui/compose/architecture>
- [36] R. Verdecchia, I. Malavolta, and P. Lago, "Guidelines for architecting android apps: A mixed-method empirical study," in *Proceedings - 2019 IEEE International Conference on Software Architecture, ICSA 2019*, 2019, doi: 10.1109/ICSA.2019.00023.